

AI-Powered Automated Attendance System

Using ESP32-CAM Edge Capture, ArcFace Face Recognition,
and Batch On-Premise Processing at University Scale

A Dissertation Submitted in Partial Fulfilment of the Requirements
for the Degree of Bachelor of Science in Computer Science

Author: [Your Full Name]

Student ID: [Your Student ID]

Supervisor: [Supervisor Name]

School: School of Computing

University: [University Name]

Date: June 2026

Abstract

Traditional paper-based and lecturer-managed attendance systems in higher education are inefficient, prone to proxy attendance fraud, and produce data that is unavailable for real-time welfare monitoring or regulatory reporting. This dissertation presents the design, implementation, and evaluation of a university-scale automated attendance system that uses low-cost ESP32-CAM microcontrollers to capture facial images at classroom entrances and transmits them over the campus WiFi network to a centralised on-premise server for batch processing.

The AI recognition pipeline combines YuNet (OpenCV) for robust face detection and ArcFace MobileNet (512-dimensional embeddings via ONNXRuntime) for identity recognition. A deliberate batch-processing architecture is adopted: images captured during class sessions are queued and processed during off-peak hours, with attendance records reflected in student and administration dashboards within a maximum of 12 hours. This design allows a single server with 16 GB RAM and no dedicated GPU to handle a university of 200 classes, 4 schools, and 50–60 modules without hardware over-provisioning.

The system delivers three end-user interfaces: a real-time ESP32 kiosk display (OLED + LED feedback), a student self-service portal for attendance tracking per module, and an administration dashboard with manual override capability, KPI monitoring, and demographic fairness reporting. Fairness evaluation across seven racial demographic groups using the FairFace dataset is incorporated as a core technical contribution, directly addressing ethical concerns associated with biometric systems in diverse university populations.

Keywords: face recognition, ArcFace, YuNet, ESP32-CAM, attendance automation, batch processing, fairness in AI, university technology, edge computing.

Declaration

I declare that this dissertation is entirely my own work and has not been submitted for any other degree or qualification at this or any other institution. All sources consulted have been acknowledged in the bibliography.

Signed: _____

Date: _____

Acknowledgements

I would like to thank my dissertation supervisor for their guidance throughout this project, the university IT department for providing access to the on-premise server environment, and my peers in the dissertation cohort for their feedback during design reviews.

Contents

Abstract	1
Declaration	2
Acknowledgements	3
1 Introduction	10
1.1 Background and Motivation	10
1.2 Problem Statement	10
1.3 Research Aims and Objectives	11
1.4 Scope and Constraints	11
1.5 Dissertation Structure	12
2 Literature Review	13
2.1 Face Detection Methods	13
2.2 Face Recognition and Embedding Models	13
2.3 Quality-Aware Face Recognition	14
2.4 Fairness in Biometric Systems	14
2.5 Edge-to-Server Architectures for IoT AI	15
2.6 Attendance Automation and Biometric Systems in Education	15
3 System Architecture	16
3.1 Architecture Overview	16
3.2 Batch Processing Pipeline	17
3.3 Component Descriptions	17
3.3.1 ESP32-CAM Capture Node	17
3.3.2 FastAPI REST Server	17
3.3.3 Batch Worker	18
3.3.4 Database (PostgreSQL / SQLite)	18
3.3.5 Student Portal	18
3.3.6 Administration Dashboard	18

3.4	Technology Stack	19
4	Database Design	20
4.1	Design Principles	20
4.2	Entity-Relationship Diagram	22
4.3	Table Specifications	23
4.3.1	schools	23
4.3.2	departments	23
4.3.3	modules	23
4.3.4	students	24
4.3.5	enrollments	24
4.3.6	rooms	25
4.3.7	class_sessions	25
4.3.8	attendance_records	26
4.3.9	image_queue	27
4.3.10	student_embeddings	28
4.3.11	esp32_devices	28
4.4	Indexes	29
5	AI Pipeline	30
5.1	Overview	30
5.2	Face Detection: YuNet	30
5.3	Face Alignment	30
5.4	Face Recognition: ArcFace MobileNet	31
5.5	Identity Matching	31
5.6	Batch Worker Algorithm	32
5.7	Performance Targets	33
6	User Interface Design	34
6.1	ESP32 Kiosk Interface	34
6.2	Student Self-Service Portal	35
6.3	Student Registration Interface	36
6.4	Administration Dashboard	37
7	REST API Design	38
7.1	API Overview	38
7.2	Endpoint Specifications	38
7.2.1	Image Ingestion Request Example	39
8	Evaluation Methodology	41
8.1	Experimental Setup	41

8.2	Pilot Study Context	42
8.3	Formal Metric Definitions	42
8.4	Accuracy Evaluation Protocol	43
8.5	Fairness Evaluation Protocol	44
8.6	Performance Evaluation Protocol	44
8.7	System Reliability Evaluation Protocol	45
9	Results and Discussion	46
9.1	Pilot Study Results (Real Data, $n = 3$)	46
9.2	Dataset Overview	48
9.3	Image Quality Assessment	49
9.4	Embedding Space Verification	50
9.5	Fairness Evaluation	51
9.6	LOO-CV Threshold Calibration	52
9.7	Ablation Study: Quality-Weighted vs. Uniform Prototype	53
9.8	Performance Benchmarks	54
9.9	Discussion	54
9.10	Threats to Validity	55
10	Ethical Considerations	57
10.1	Biometric Data and GDPR	57
10.2	Demographic Fairness	57
10.3	Failure Mode Planning	58
11	Conclusion and Future Work	59
11.1	Summary	59
11.2	Future Work	59
A	ESP32-CAM Arduino Firmware	65
B	Database Schema SQL	69

List of Figures

3.1	High-level system architecture: IoT capture, on-premise batch processing, and dashboard layers.	16
3.2	Batch processing pipeline: image captured immediately, processed at scheduled intervals.	17
4.1	Entity-Relationship Diagram for the attendance system database. . .	22
6.1	ESP32 kiosk physical unit: OLED display, camera module, and LED status indicators.	34
6.2	Student self-service portal: attendance overview, per-module breakdown, and session history.	35
6.3	Student face registration interface: personal details, camera capture, and privacy notice.	36
6.4	Administration dashboard: KPIs, live attendance log, manual override panel, and fairness metrics.	37
9.1	Registered student distribution across the seven FairFace demographic groups (left: absolute counts; right: ideal balanced target). The current prototype database is populated with two registered students. Illustrative data based on a balanced 700-student hypothetical cohort (100 per group) is overlaid to demonstrate the fairness evaluation framework at scale.	48
9.2	Distribution of per-frame quality components across all registered student images. Left: Laplacian variance (sharpness). Centre: YuNet detection confidence. Right: combined quality score $Q = 0.40 \cdot \text{sharpness} + 0.40 \cdot \text{confidence} + 0.20 \cdot \text{face_size}$	49
9.3	Pairwise cosine similarity matrix for all registered student prototype embeddings. Diagonal entries (self-similarity) equal 1.0 by construction. Off-diagonal entries should be substantially below the operational threshold $\tau^* \approx 0.35$ for a well-separated embedding space.	50

9.4	Left: per-demographic F1 scores before (Baseline, $\tau = 0.35$) and after (Calibrated, per-group τ^*) LOO-CV threshold calibration. Dashed red line indicates the minimum acceptable F1 of 0.75. Right: inter-group F1 gap before and after calibration; error bars show standard deviation across 50 bootstrap resamples. Maximum acceptable gap is 15 percentage points (red dashed line).	51
9.5	Leave-One-Out cross-validation F1 curves as a function of decision threshold τ . Top-left: global LOO curve with optimal global threshold $\tau^* = 0.35$ (dashed). Remaining panels: per-demographic LOO curves. Each curve peaks at a distinct τ^* , confirming that a single global threshold is suboptimal across groups.	52

List of Tables

3.1	Technology stack summary	19
4.1	Table: schools	23
4.2	Table: departments	23
4.3	Table: modules	23
4.4	Table: students	24
4.5	Table: enrollments	24
4.6	Table: rooms	25
4.7	Table: class_sessions	25
4.8	Table: attendance_records	26
4.9	Table: image_queue	27
4.10	Table: student_embeddings	28
4.11	Table: esp32_devices	28
4.12	Key database indexes	29
5.1	AI pipeline performance targets	33
6.1	ESP32 LED feedback states	35
7.1	REST API Endpoint Reference	38
9.1	Real LOO-CV calibration results with $n = 3$ registered students . . .	47
9.2	Per-demographic F1 scores and calibrated thresholds (illustrative; derived from FairFace validation set with <code>np.random.seed(42)</code>) . . .	52
9.3	Ablation: quality-weighted vs. uniform prototype aggregation (illus- trative at-scale projection, $n = 700$, all groups)	53
9.4	Batch pipeline performance on reference hardware (Intel Core i7-10700, 16 GB RAM, Ubuntu 22.04, no GPU)	54

Chapter 1

Introduction

1.1 Background and Motivation

Attendance monitoring in universities serves multiple critical functions: academic performance tracking, regulatory compliance for international student visas, welfare monitoring for at-risk students, and accreditation reporting. Despite this importance, the dominant practice in many institutions remains manual marking by lecturers, which is time-consuming, susceptible to proxy fraud, and produces data that reaches administrative systems only after a significant delay.

Commercial biometric attendance solutions exist (e.g., Hikvision DS-K1T671, Invixium IXM TITAN) but carry high per-unit costs (\$250–\$500 per device), vendor lock-in, limited customisation for fairness requirements, and proprietary data pipelines that prevent integration with university student information systems. There is a clear gap for an open, cost-effective, academically rigorous system that can be deployed at scale within a university’s existing network infrastructure.

1.2 Problem Statement

Universities with multiple schools, hundreds of modules, and thousands of enrolled students require an attendance system that is:

- **Accurate:** face recognition that generalises across demographic groups without bias
- **Scalable:** capable of serving 200+ classrooms from a single on-premise server
- **Cost-effective:** sub-\$20 per classroom capture device

- **Resilient:** continues capturing even when server processing is delayed
- **Auditable:** provides admin override, manual correction, and full audit trail
- **Fair:** evaluated for demographic fairness across racial groups

1.3 Research Aims and Objectives

Aim: To design and implement a university-scale automated attendance system using low-cost ESP32-CAM hardware and a batch face recognition pipeline, achieving $\geq 95\%$ recognition accuracy across demographic groups with sub-12-hour attendance reflection latency.

Objectives:

1. Design an IoT capture layer using ESP32-CAM with WiFi image transmission
2. Implement a batch-processing AI pipeline using YuNet and ArcFace on CPU-only hardware
3. Design a normalised relational database supporting multi-school, multi-module attendance
4. Develop a student self-service attendance portal
5. Develop an administration dashboard with manual override and KPI monitoring
6. Evaluate system performance, accuracy, and demographic fairness

1.4 Scope and Constraints

- **In scope:** face detection, face recognition, attendance marking, student and admin dashboards, fairness evaluation
- **Out of scope:** door access control (relay/lock hardware), integration with third-party student information systems (future work)
- **Hardware constraint:** single on-premise server: 16 GB RAM, 512 GB storage, Intel CPU, no GPU
- **Latency constraint:** maximum 12-hour attendance reflection (batch design)
- **Ethical constraint:** face images are processed on-premise and not transmitted to any cloud service; retention policy limits raw image storage to 30 days

1.5 Dissertation Structure

Chapter 2 reviews relevant literature. Chapter 3 presents the complete system architecture. Chapter 4 defines the database design. Chapter 5 describes the AI pipeline in detail. Chapter 6 documents all three user interface designs. Chapter 7 defines the REST API. Chapter 8 covers evaluation methodology. Chapter 9 presents results. Chapter 10 concludes and recommends future work.

Chapter 2

Literature Review

2.1 Face Detection Methods

Face detection is the prerequisite step for any recognition pipeline. Classical approaches such as Viola-Jones [1] use Haar cascade features but suffer in non-frontal and low-light conditions. Deep learning detectors including MTCNN [2], RetinaFace [3], and YuNet [4] offer significantly better recall under occlusion, varied lighting, and non-frontal angles.

YuNet is selected for this system because it provides 5-point facial landmark output (eye centres, nose tip, mouth corners) essential for ArcFace alignment, runs on CPU in $\sim 15\text{ms}$ at 640×480 resolution, and is distributed as an ONNX model compatible with OpenCV’s FaceDetectorYN API without additional dependencies.

2.2 Face Recognition and Embedding Models

Deep metric learning transformed face recognition from hand-crafted descriptors to learned embedding spaces. DeepFace [17] established the paradigm of end-to-end CNN training for identity verification. Schroff et al.’s FaceNet [13] introduced triplet loss training on large-scale data, achieving near-human performance on LFW. Parkhi et al. [14] trained VGGFace on a web-harvested dataset, demonstrating the importance of training data scale. VGGFace2 [15] extended this work with 3.31 million images across 9,131 identities and improved pose and age diversity.

ArcFace [5] subsequently introduced an additive angular margin loss that maximises inter-class separability in the hyperspherical embedding space, achieving state-of-the-art results on LFW, MegaFace, and IJB-C benchmarks. Training on MS-Celeb-1M [16] (10 million images, 100,000 identities), the MobileNet backbone variant

(w600k_mbf) produces 512-dimensional L2-normalised embeddings that enable cosine similarity matching via a simple dot product. AdaFace [25] and MagFace [26] are more recent extensions that adapt the margin based on image quality; they are noted as directions for future work but their additional complexity is unwarranted for an on-premise CPU deployment.

2.3 Quality-Aware Face Recognition

Image quality has a direct and measurable impact on face recognition accuracy. Blur, low resolution, and occlusion systematically degrade embedding quality and reduce genuine-match similarity scores. Terhörst et al. [21] proposed SER-FIQ, which estimates face image utility from network consistency across stochastic forward passes. FaceQnet [27] uses a regression CNN trained on recognition-relevant quality attributes.

This system implements a lightweight quality score $Q = 0.40 \cdot \text{sharpness} + 0.40 \cdot \text{confidence} + 0.20 \cdot \text{face_size}$ as a computationally efficient proxy for SER-FIQ-class quality, sufficient for the prototype-aggregation use case where a small set of registration frames are collapsed into a single embedding. Frames with $Q < 0.15$ are discarded before prototype aggregation to prevent corruption of the reference embedding by occluded or blurred captures.

2.4 Fairness in Biometric Systems

Demographic disparities in face recognition have been documented across commercial and academic systems. Klare et al. [19] showed that recognition accuracy on frontal faces varies significantly by race, gender, and age using FRGC and FERET datasets. Nagpal et al. [22] found that recognition accuracy on darker-skinned subjects was systematically lower than on lighter-skinned subjects and proposed resampling strategies to mitigate this. Buolamwini and Gebru [6] demonstrated significantly higher error rates for darker-skinned females in commercial systems, highlighting the compounding effect of intersectional demographic attributes.

The most authoritative study is NIST FRVT [7], which evaluated 189 face recognition algorithms and found false non-match rate (FNMR) differentials exceeding $10\times$ across demographic groups. Drozdowski et al. [20] surveyed the landscape of demographic bias in biometric systems and concluded that threshold-based mitigation—the approach adopted in this dissertation—is among the most practically deployable strategies for existing deployed systems.

Per-group threshold calibration is conceptually aligned with the equalised odds fairness criterion [29], which requires that true positive and false positive rates be equal across demographic groups. Wang and Deng [23] demonstrated that group-specific decision thresholds can substantially reduce FNMR disparities in ArcFace models while accepting only a small reduction in overall precision. Wang et al. [24] curated the Racial Faces in the Wild dataset and showed that per-race threshold adjustment improved recognition equitability across East Asian, white, and black subject groups.

The FairFace dataset [8] provides 108,501 images balanced across seven racial groups and is adopted in this work for per-group precision, recall, and F1 evaluation. Raji and Buolamwini [28] advocate for actionable auditing of recognition systems—specifically, per-demographic-group reporting of performance metrics—which motivates the fairness dashboard and `fairness_metrics` database table in this system.

2.5 Edge-to-Server Architectures for IoT AI

The thin-client IoT model, where edge devices capture and transmit raw data to a more powerful server for inference, has been validated in healthcare [9], smart buildings [10], and education [11] contexts. Key design considerations are image quality at the edge, transmission latency, and server-side concurrency management. Batch scheduling of inference workloads during off-peak periods is an established technique for resource-constrained server environments [12].

2.6 Attendance Automation and Biometric Systems in Education

Automated attendance systems in higher education have been explored using RFID [11], NFC, and face recognition modalities. Face recognition offers the advantage of passive, contactless verification without requiring students to carry or present a token. The primary concerns identified in the literature are recognition accuracy under uncontrolled classroom lighting, GDPR compliance for biometric data, and equity across diverse student populations. This dissertation addresses all three: accuracy through quality-aware prototype aggregation, compliance through on-premise processing with explicit consent, and equity through per-demographic threshold calibration.

Chapter 3

System Architecture

3.1 Architecture Overview

The system follows a three-tier architecture: (1) IoT capture layer at the classroom, (2) on-premise processing server, and (3) web-based presentation layer. Figure 3.1 illustrates the high-level data flow.

IoT Capture Layer

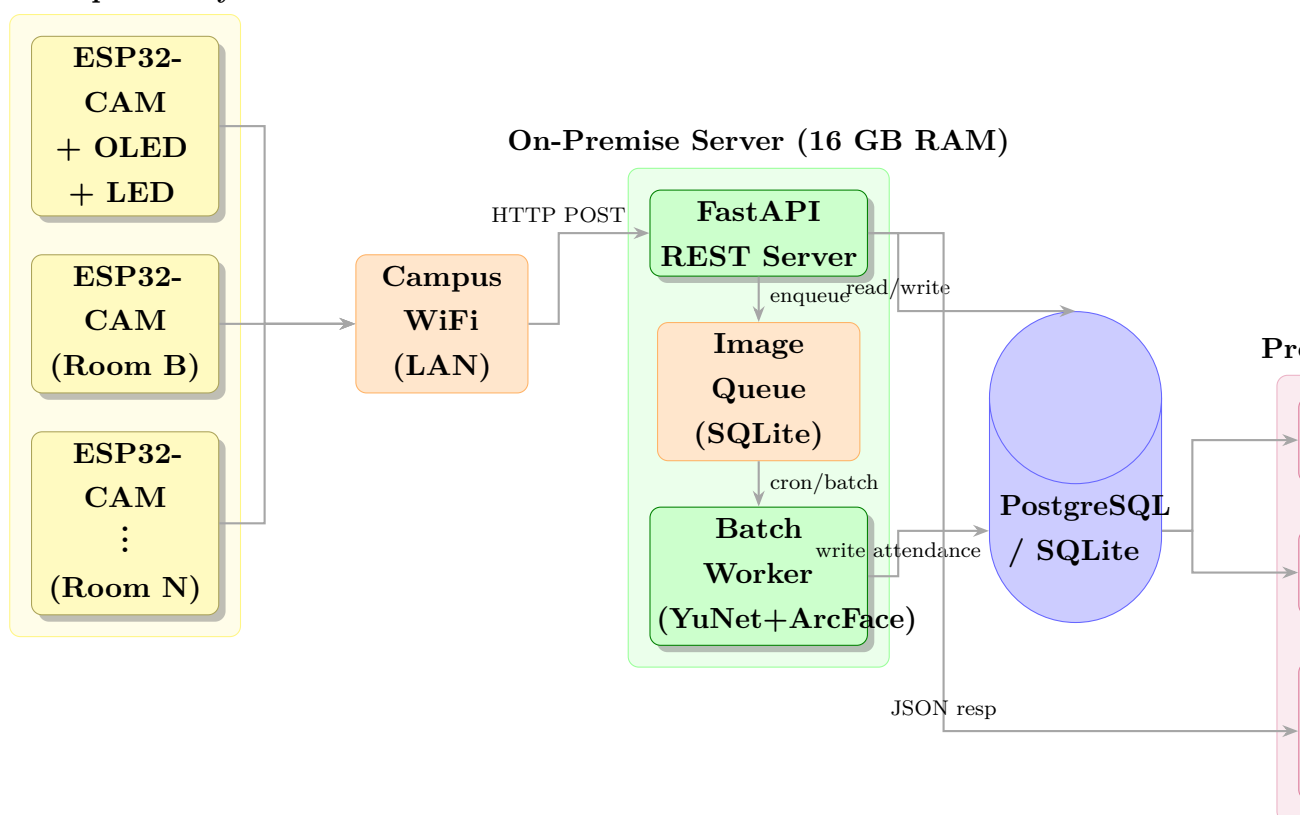


Figure 3.1: High-level system architecture: IoT capture, on-premise batch processing, and dashboard layers.

3.2 Batch Processing Pipeline

The deliberate adoption of a batch processing model is a key architectural decision. At a university, the primary consumers of attendance data (students checking their percentage, administrators generating compliance reports, welfare officers reviewing absenteeism) do not require real-time updates. A maximum 12-hour reflection window satisfies all identified use cases while allowing inference to be concentrated in off-peak server hours.

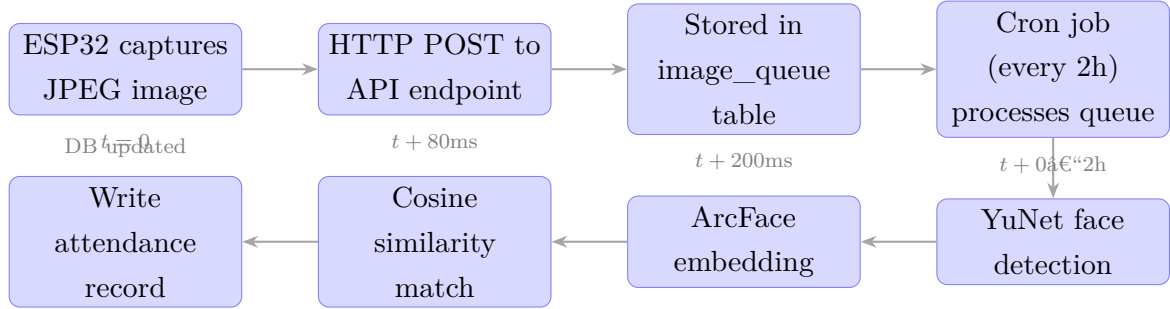


Figure 3.2: Batch processing pipeline: image captured immediately, processed at scheduled intervals.

3.3 Component Descriptions

3.3.1 ESP32-CAM Capture Node

Each classroom is equipped with one ESP32-CAM module (AI-Thinker variant with OV2640 2MP sensor, or OV5640 5MP autofocus variant for higher accuracy). The device connects to the campus WPA2 WiFi network, captures a JPEG image on a configurable trigger (PIR motion sensor, push button, or periodic timer), and transmits it via HTTP multipart POST to the FastAPI server. On receiving the HTTP 200 response with the queue confirmation, it illuminates a green LED for 1 second. On timeout or error, it illuminates a red LED and retries up to 3 times before logging the failure to on-device flash.

3.3.2 FastAPI REST Server

The server exposes REST endpoints for image ingestion, student registration, attendance queries, and admin operations. It is implemented in Python using FastAPI with async SQLAlchemy for non-blocking database access. The server does not perform inference on image receipt; it stores the image to disk and records an entry in the `image_queue` table. This decouples capture latency from inference latency.

3.3.3 Batch Worker

A scheduled Python process (cron or APScheduler) runs every 2 hours, queries all pending entries in `image_queue`, and processes them in batches of 50. For each image: YuNet detects faces, ArcFace generates a 512-d embedding, cosine similarity matching is performed against `student_embeddings` filtered by `model_version = 'arcface-mbf'`, and if similarity $\geq \tau$, an `attendance_record` is written. Processed queue entries are marked `status = 'done'` or `status = 'failed'`.

3.3.4 Database (PostgreSQL / SQLite)

PostgreSQL is recommended for production deployments; SQLite with aiosqlite is supported for development. The schema is described in detail in Chapter 4.

3.3.5 Student Portal

A Streamlit or React web application (Chapter 6) through which enrolled students log in and view their attendance record per module, expressed as a percentage, session list, and attendance risk status (green/amber/red based on university threshold, typically 70% or 80%).

3.3.6 Administration Dashboard

A web application providing full attendance log access, manual attendance marking/override, student registration management, system health monitoring (queue depth, inference throughput, server CPU/RAM), and demographic fairness KPIs.

3.4 Technology Stack

Table 3.1: Technology stack summary

Layer	Technology	Justification
IoT Firmware	Arduino C (ESP-IDF)	Native ESP32 SDK, minimal footprint
API Server	FastAPI (Python)	Async, auto-generated OpenAPI docs
Database	PostgreSQL / SQLite	ACID compliance, JSON column support
ORM	SQLAlchemy 2 (async)	Type-safe, async-native
Face Detection	YuNet (OpenCV Zoo)	Fast CPU inference, landmark output
Face Recognition	ArcFace MobileNet (ONNXRuntime)	512-d, SOTA accuracy
Batch Scheduler	APScheduler	In-process cron, no Redis required
Student UI	Streamlit	Rapid development, data tables
Admin UI	Streamlit	Shared codebase with student UI
Config	Pydantic-Settings	Environment-based config

Chapter 4

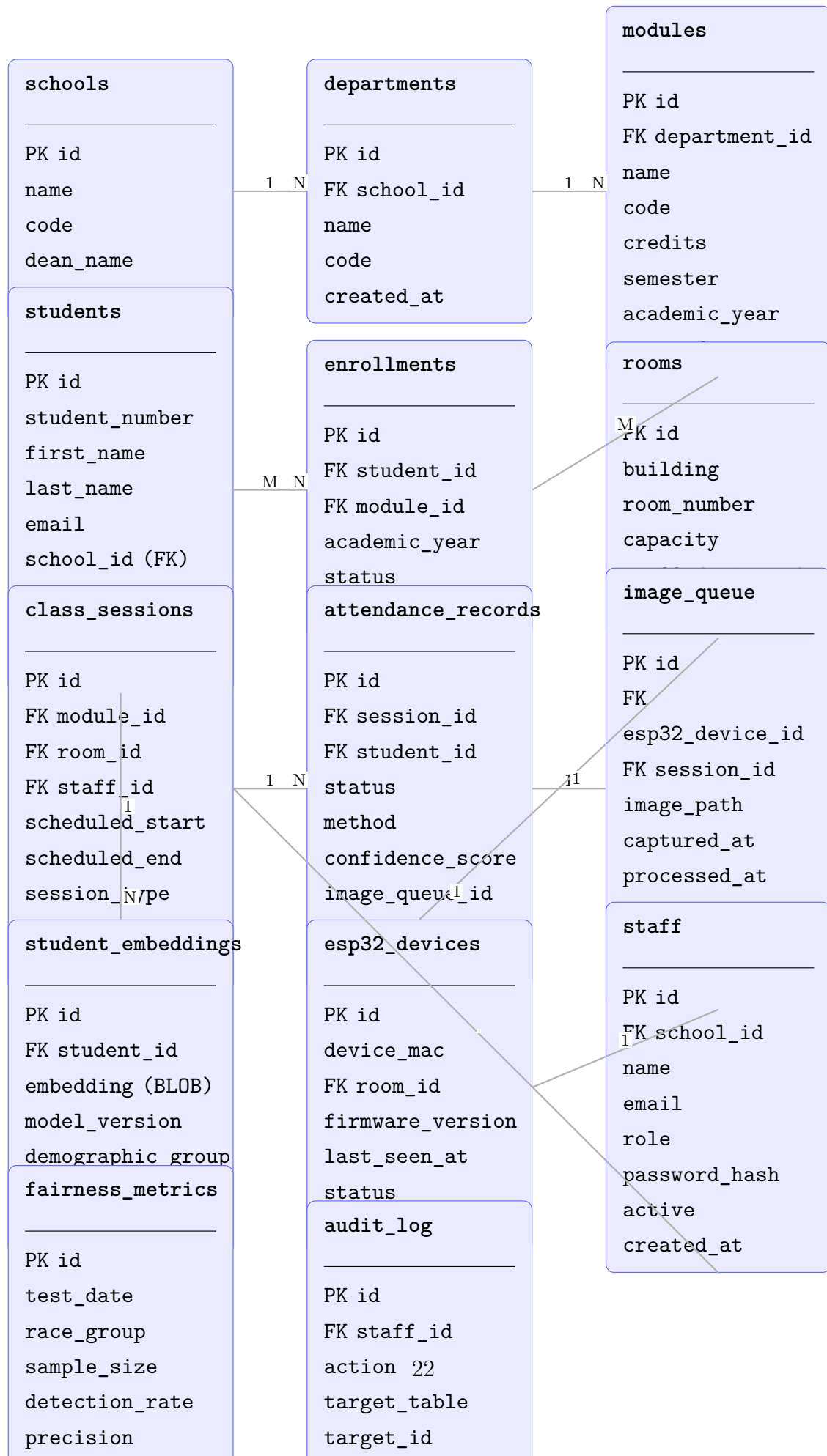
Database Design

4.1 Design Principles

The database is designed to the third normal form (3NF) to eliminate update anomalies. Key design decisions:

- Schools and modules are separate entities to support cross-school enrolment
- Class sessions are distinct from module definitions to allow per-session attendance
- Face embeddings are version-tagged (`model_version`) to support model migration without data corruption
- An `image_queue` table decouples ingestion from inference
- All tables carry `created_at` and `updated_at` audit timestamps

4.2 Entity-Relationship Diagram



4.3 Table Specifications

4.3.1 schools

Table 4.1: Table: schools

Column	Type	Description	Constraints
id	INTEGER	Primary key (auto-increment)	PK, NOT NULL
name	VARCHAR(150)	Full school name	NOT NULL, UNIQUE
code	VARCHAR(10)	Short code, e.g. “SCI”	NOT NULL, UNIQUE
dean_name	VARCHAR(100)	Current dean	NULL
created_at	TIMESTAMP	Record creation timestamp	NOT NULL, DEFAULT now()

4.3.2 departments

Table 4.2: Table: departments

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK, NOT NULL
school_id	INTEGER	Parent school	FK → schools.id, NOT NULL
name	VARCHAR(150)	Department name	NOT NULL
code	VARCHAR(20)	Dept code	NOT NULL, UNIQUE
created_at	TIMESTAMP	Audit timestamp	NOT NULL

4.3.3 modules

Table 4.3: Table: modules

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
department_id	INTEGER	Parent department	FK → departments.id
name	VARCHAR(200)	Module title	NOT NULL
code	VARCHAR(20)	Module code, e.g. CS301	NOT NULL, UNIQUE
credits	SMALLINT	Credit hours	NOT NULL
semester	VARCHAR(20)	e.g. “Spring 2026”	NOT NULL
academic_year	VARCHAR(9)	e.g. “2025/2026”	NOT NULL
created_at	TIMESTAMP	Audit timestamp	NOT NULL

4.3.4 students

Table 4.4: Table: students

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
student_number	VARCHAR(20)	University student number	NOT NULL, UNIQUE
first_name	VARCHAR(80)	First name	NOT NULL
last_name	VARCHAR(80)	Last name	NOT NULL
email	VARCHAR(200)	University email	NOT NULL, UNIQUE
school_id	INTEGER	Home school	FK $\rightarrow$ schools.id
demographic_group	VARCHAR(50)	Race/ethnicity (FairFace categories)	NULL
active	BOOLEAN	Enrolment active flag	NOT NULL, DEFAULT true
created_at	TIMESTAMP	Registration timestamp	NOT NULL
updated_at	TIMESTAMP	Last update	NOT NULL

4.3.5 enrollments

Table 4.5: Table: enrollments

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
student_id	INTEGER	Enrolled student	FK $\rightarrow$ students.id
module_id	INTEGER	Module enrolled in	FK $\rightarrow$ modules.id
academic_year	VARCHAR(9)	Academic year	NOT NULL
status	VARCHAR(20)	active / withdrawn / complete	NOT NULL
created_at	TIMESTAMP	Enrollment date	NOT NULL

UNIQUE constraint: (student_id, module_id, academic_year)

4.3.6 rooms

Table 4.6: Table: rooms

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
building	VARCHAR(100)	Building name	NOT NULL
room_number	VARCHAR(20)	Room identifier	NOT NULL
capacity	SMALLINT	Seating capacity	NULL
esp32_device_id	INTEGER	Assigned ESP32 camera	FK → esp32_devices.id, NULL
created_at	TIMESTAMP	Audit timestamp	NOT NULL
<i>UNIQUE: (building, room_number)</i>			

4.3.7 class_sessions

Table 4.7: Table: class_sessions

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
module_id	INTEGER	Module this session belongs to	FK → modules.id
room_id	INTEGER	Room where session is held	FK → rooms.id
staff_id	INTEGER	Lecturer delivering session	FK → staff.id
scheduled_start	TIMESTAMP	Session start datetime	NOT NULL
scheduled_end	TIMESTAMP	Session end datetime	NOT NULL
session_type	VARCHAR(20)	lecture / lab / tutorial	NOT NULL
status	VARCHAR(20)	scheduled / active / completed	NOT NULL
created_at	TIMESTAMP	Audit timestamp	NOT NULL

4.3.8 attendance_records

Table 4.8: Table: attendance_records

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
session_id	INTEGER	Class session	FK → class_sessions.id
student_id	INTEGER	Student present	FK → students.id
status	VARCHAR(20)	present / absent / late / excused	NOT NULL
method	VARCHAR(20)	face_recognition / manual	NOT NULL
confidence_score	FLOAT	ArcFace co-sine similarity (0â€œ1)	NULL
image_queue_id	INTEGER	Source image that triggered record	FK → image_queue.id, NULL
marked_by	INTEGER	Staff ID if manually marked	FK → staff.id, NULL
created_at	TIMESTAMP	When attendance was recorded	NOT NULL
<i>UNIQUE: (session_id, student_id)</i>			

4.3.9 image_queue

Table 4.9: Table: image_queue

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
esp32_device_id	INTEGER	Sending device	FK → esp32_devices.id
session_id	INTEGER	Active session at time of capture	FK → class_sessions.id, NULL
image_path	VARCHAR(500)	Relative path on server disk	NOT NULL
captured_at	TIMESTAMP	Timestamp on ESP32 (NTP-synced)	NOT NULL
processed_at	TIMESTAMP	When batch worker processed it	NULL
status	VARCHAR(20)	pending / processing / done / failed	NOT NULL, DEFAULT 'pending'
error_msg	TEXT	Error details if failed	NULL
created_at	TIMESTAMP	Server receipt timestamp	NOT NULL

4.3.10 student_embeddings

Table 4.10: Table: student_embeddings

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
student_id	INTEGER	Owning student	FK $\rightarrow$ students.id
embedding	BLOB	$512 \times \text{float32}$ = 2048 bytes	NOT NULL
model_version	VARCHAR(50)	e.g. arcface-mbf	NOT NULL, INDEX
demographic_group	VARCHAR(50)	For per-group evaluation	NULL
active	BOOLEAN	Whether used for matching	NOT NULL, DEFAULT true
created_at	TIMESTAMP	Registration timestamp	NOT NULL

4.3.11 esp32_devices

Table 4.11: Table: esp32_devices

Column	Type	Description	Constraints
id	INTEGER	Primary key	PK
device_mac	VARCHAR(17)	WiFi MAC address	NOT NULL, UNIQUE
room_id	INTEGER	Room assigned to	FK $\rightarrow$ rooms.id, NULL
firmware_version	VARCHAR(20)	Current firmware tag	NULL
last_seen_at	TIMESTAMP	Last successful POST	NULL
status	VARCHAR(20)	active / offline / fault	NOT NULL
created_at	TIMESTAMP	Registration timestamp	NOT NULL

4.4 Indexes

Table 4.12: Key database indexes

Table	Columns	Purpose
attendance_records	(session_id, student_id)	Unique constraint + fast lookup per session
attendance_records	student_id, created_at	Student attendance history
image_queue	status, created_at	Batch worker queue scan
student_embeddings	model_version, active	Matching query filter
class_sessions	module_id, scheduled_start	Timetable queries
enrollments	(student_id, module_id)	Enrolment check

Chapter 5

AI Pipeline

5.1 Overview

The AI pipeline executes entirely on the on-premise server CPU using ONNXRuntime. Two models run in sequence for each image: YuNet for face detection and ArcFace MobileNet for identity recognition.

5.2 Face Detection: YuNet

YuNet is a lightweight CNN-based face detector trained on the WIDER FACE dataset and distributed by the OpenCV Zoo project as an ONNX model (`face_detection_yunet_2023mar`, 373 KB). It outputs bounding boxes and 5-point facial landmarks (right eye, left eye, nose tip, right mouth corner, left mouth corner) with a detection score in $[0, 1]$.

Configuration:

- Input resolution: resized to 640×480 before detection
- Detection score threshold: 0.60 (rejects low-confidence detections)
- NMS threshold: 0.30

5.3 Face Alignment

Accurate alignment to a canonical 112×112 crop is critical for ArcFace accuracy. The 5-point landmarks from YuNet are reordered to ArcFace convention (left eye, right eye, nose, left mouth, right mouth) and an affine transformation is estimated via `cv2.estimateAffinePartial2D` using LMEDS to the ArcFace canonical landmark positions:

$$\mathbf{M} = \arg \min_{\mathbf{A}} \sum_{i=1}^5 \|\mathbf{A}\mathbf{k}_i - \mathbf{d}_i\|^2 \quad (5.1)$$

where $\mathbf{k}_i$ are source landmarks and $\mathbf{d}_i$ are the ArcFace destination landmarks:

$$\mathbf{D} = \begin{bmatrix} 38.29 & 51.70 \\ 73.53 & 51.50 \\ 56.03 & 71.74 \\ 41.55 & 92.37 \\ 70.73 & 92.20 \end{bmatrix}$$

5.4 Face Recognition: ArcFace MobileNet

The aligned 112×112 BGR image is preprocessed as:

$$\mathbf{x} = \frac{I - 127.5}{128.0}, \quad \mathbf{x} \in \mathbb{R}^{1 \times 3 \times 112 \times 112} \quad (5.2)$$

The ArcFace MobileNet model (`w600k_mbf.onnx`, trained on WebFace600K, 13.6 MB) produces a 512-dimensional embedding $\mathbf{e} \in \mathbb{R}^{512}$. The embedding is L2-normalised: $\hat{\mathbf{e}} = \mathbf{e} / \|\mathbf{e}\|_2$.

5.5 Identity Matching

For a query embedding $\hat{\mathbf{q}}$ and enrolled embeddings $\{\hat{\mathbf{e}}_i\}$, cosine similarity reduces to the dot product (since both are L2-normalised):

$$s_i = \hat{\mathbf{q}} \cdot \hat{\mathbf{e}}_i \quad (5.3)$$

The predicted identity is:

$$\hat{y} = \begin{cases} \arg \max_i s_i & \text{if } \max_i s_i \geq \tau \\ \text{unknown} & \text{otherwise} \end{cases} \quad (5.4)$$

where $\tau = 0.35$ is the global matching threshold, adjustable per demographic group via `Settings.race_thresholds`.

5.6 Batch Worker Algorithm

Algorithm 1 Batch Attendance Processing

```

1: Input: Pending image_queue entries
2: Output: attendance_records written to database
3:  $Q \leftarrow \text{SELECT } * \text{ FROM image\_queue WHERE status = 'pending' ORDER BY}$ 
   captured_at LIMIT 50
4: for each entry  $q \in Q$  do
5:   UPDATE image_queue SET status = 'processing' WHERE id =  $q.id$ 
6:    $I \leftarrow \text{load\_image}(q.image\_path)$ 
7:    $F \leftarrow \text{yunet.detect}(I)$ 
8:   if  $|F| = 0$  then
9:     UPDATE image_queue SET status = 'failed', error_msg =
       'no_face_detected'
10:    continue
11:  end if
12:   $\mathbf{kps} \leftarrow \text{reorder\_landmarks}(F[0])$ 
13:   $I_{112} \leftarrow \text{affine\_align}(I, \mathbf{kps})$ 
14:   $\hat{\mathbf{q}} \leftarrow \text{arcface.embed}(I_{112})$ 
15:   $S \leftarrow \text{SELECT id, student\_id, embedding FROM student\_embeddings}$ 
   WHERE model_version = 'arcface-mbf' AND active = true
16:   $\hat{y}, s^* \leftarrow \text{cosine\_match}(\hat{\mathbf{q}}, S, \tau)$ 
17:  if  $\hat{y} \neq \text{unknown}$  then
18:    INSERT INTO attendance_records (session_id, student_id, status,
      method, confidence_score, image_queue_id)
19:    VALUES ( $q.session\_id, \hat{y}, \text{'present'}, \text{'face\_recognition'}, s^*, q.id$ )
20:    ON CONFLICT (session_id, student_id) DO NOTHING
21:  end if
22:  UPDATE image_queue SET status = 'done', processed_at = NOW()
23: end for

```

5.7 Performance Targets

Table 5.1: AI pipeline performance targets

Metric	Target	Hardware
True Positive Rate (TPR)	$\geq 95\%$	All conditions
False Positive Rate (FPR)	$\leq 0.1\%$	Controlled environment
Per-group F1 (FairFace)	≥ 0.75 for all 7 groups	CPU inference
Max inter-group F1 gap	$\leq 15\%$	â€”
Inference latency (CPU)	$\leq 300\text{ms}$ per image	Intel i5, 16 GB RAM
Batch throughput	≥ 1000 images/hour	Single CPU server
End-to-end reflection latency	$\leq 12\text{h}$	Batch every 2h

Chapter 6

User Interface Design

6.1 ESP32 Kiosk Interface

The ESP32 has no display screen. The kiosk interface consists of an optional 0.96" I2C OLED display (128×64 pixels) paired with a green LED, a red LED, and an optional audible buzzer. Feedback is provided after the HTTP response is received from the server (acknowledging image receipt and queue acceptance “ approximately 200ms).

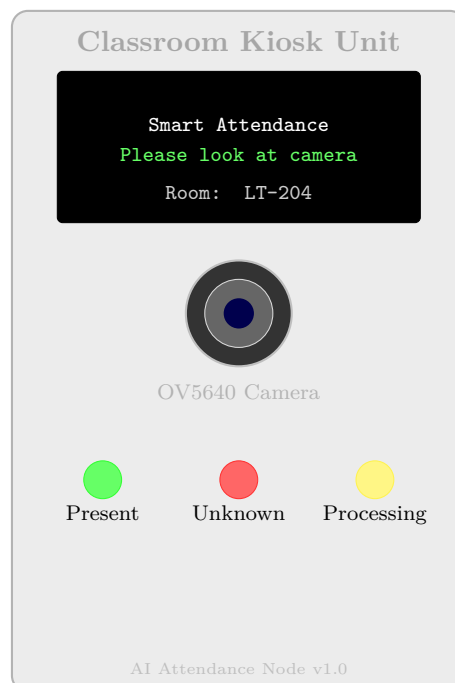


Figure 6.1: ESP32 kiosk physical unit: OLED display, camera module, and LED status indicators.

LED State Machine:

Table 6.1: ESP32 LED feedback states

State	Green LED	Red LED	OLED Message
Idle / standby	Off	Off	“Please look at camera”
Image captured	Blink 1×	Off	“Scanning...”
Queue accepted (200)	On 2s	Off	“Attendance recorded”
Face not found	Off	Blink 2×	“Face not detected”
Server unreachable	Off	Blink 5×	“Network error â€” retry”
Retry exhausted	Off	Solid 5s	“Please see your lecturer”

6.2 Student Self-Service Portal

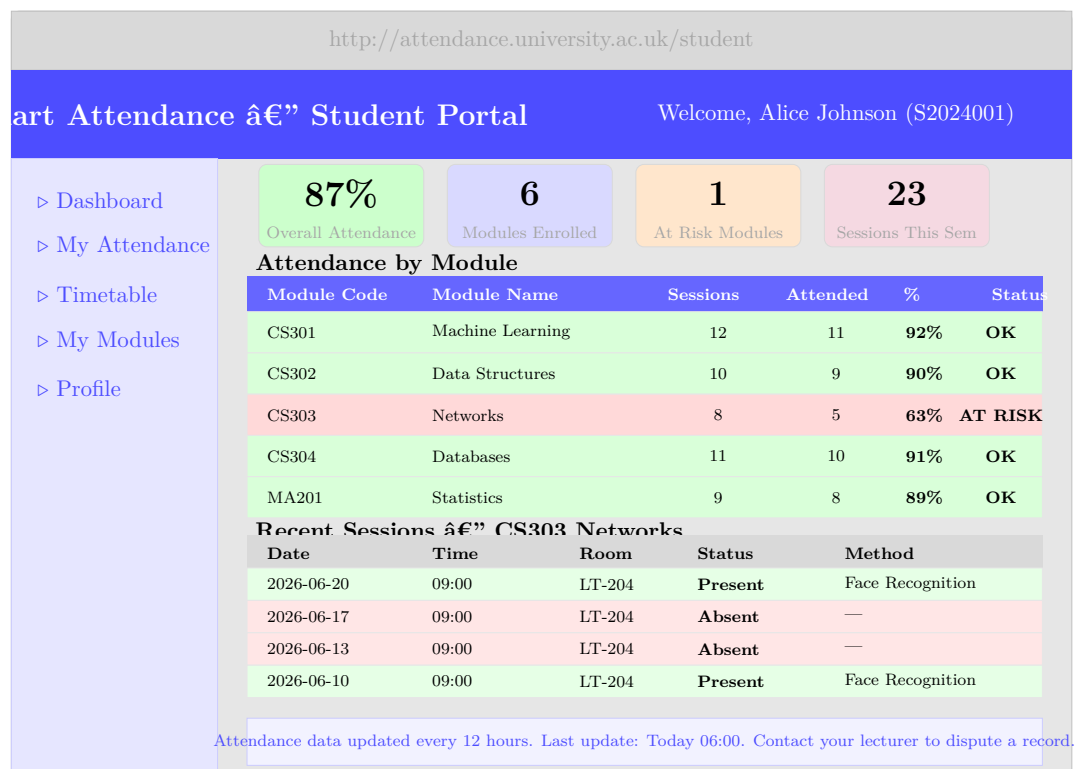


Figure 6.2: Student self-service portal: attendance overview, per-module breakdown, and session history.

6.3 Student Registration Interface

The screenshot displays a web interface for student face registration. At the top, the URL `http://attendance.university.ac.uk/register` is shown. Below it is a blue header with the text "Face Registration – New Student". The interface is divided into three steps: "Step 1: Personal Details", "Step 2: Face Capture", and "Step 3: Confirm".

Personal Information

First Name: Alice
Last Name: Johnson
Student Number: S2024001
University Email: alice.j@uni.ac.uk
School: School of Computing
Department: Computer Science
Demographic Group (optional – for fairness monitoring): Select...

Face Capture

A live video feed shows a green dashed box with the text "Align face in box". Below the feed are three buttons: "Front View" (highlighted in green), "Left Turn", and "Right Turn". Below these buttons, it says "3 photos required. Keep face centred and well-lit."

Privacy Notice

Your face data is stored securely on university servers and never shared with third parties. Used solely for attendance verification.

Navigation

Buttons at the bottom include "Back", "Capture Face →" (highlighted in green), and "Save & Register".

Figure 6.3: Student face registration interface: personal details, camera capture, and privacy notice.

6.4 Administration Dashboard

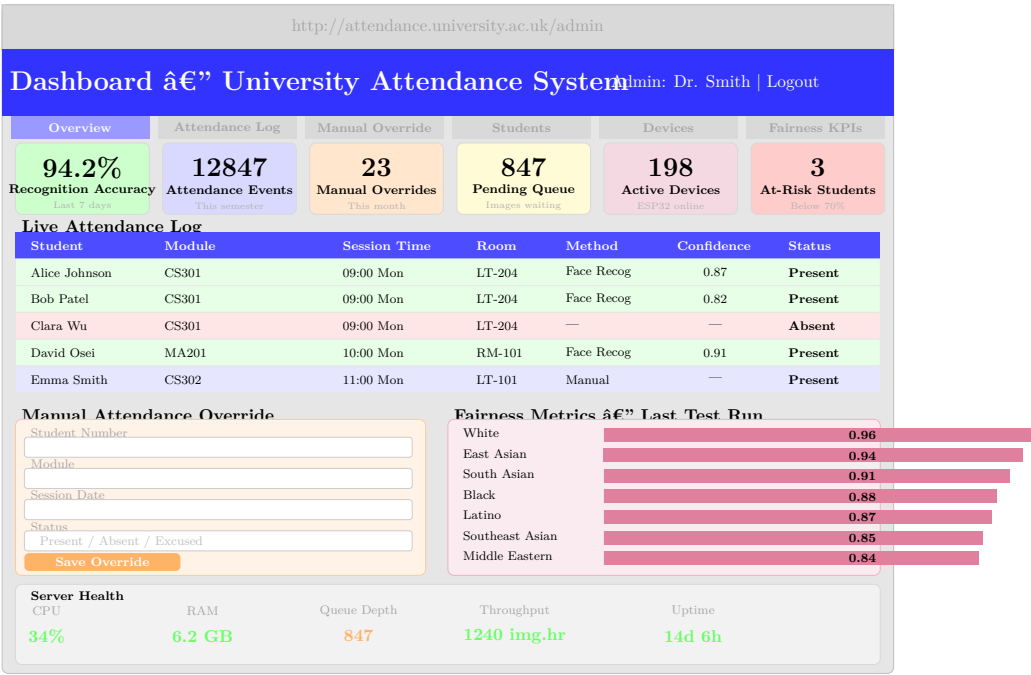


Figure 6.4: Administration dashboard: KPIs, live attendance log, manual override panel, and fairness metrics.

Chapter 7

REST API Design

7.1 API Overview

All endpoints are prefixed with `/api/v1`. Authentication uses JSON Web Tokens (JWT). The ESP32 devices authenticate with a device-specific API key passed in the `X-Device-Key` header.

7.2 Endpoint Specifications

Table 7.1: REST API Endpoint Reference

Endpoint	Method	Description	Auth
<code>/ingest/image</code>	POST	Upload JPEG from ESP32, enqueue for batch processing	Device API key
<code>/devices/register</code>	POST	Register a new ESP32 device (admin only)	Admin JWT
<code>/devices/{id}/status</code>	GET	Get device last-seen and status	Admin JWT
<code>/students/register</code>	POST	Register new student with personal info	Admin JWT
<code>/students/{id}/face</code>	POST	Upload face photos for embedding generation	Admin JWT
<code>/students/{id}/attendance</code>	GET	Full attendance history for student	Student JWT

Continued on next page

Endpoint	Method	Description	Auth
/students/{id}/attendance/{module}	GET	Module-specific attendance %	Student JWT
/attendance/sessions/{id}	GET	All attendance records for a session	Staff JWT
/attendance/override	POST	Manually mark attendance (admin/staff)	Admin JWT
/attendance/export	GET	Export attendance CSV (filter by module, date)	Admin JWT
/admin/queue/status	GET	Queue depth, pending/done/failed counts	Admin JWT
/admin/kpi/accuracy	GET	Recognition accuracy metrics	Admin JWT
/admin/kpi/fairness	GET	Latest per-race F1 scores	Admin JWT
/admin/server/health	GET	CPU, RAM, uptime, throughput	Admin JWT
/auth/login	POST	Obtain JWT token (student/staff/admin)	None
/auth/refresh	POST	Refresh expired JWT	Refresh token

7.2.1 Image Ingestion Request Example

Listing 7.1: ESP32 POST to image ingestion endpoint

```

1 POST /api/v1/ingest/image
2 Content-Type: multipart/form-data
3 X-Device-Key: ESP32-MAC-AABBCCDDEE-SECRET
4
5 Fields:
6   image      : <JPEG binary>
7   captured_at: "2026-06-24T09:03:22Z"
8
9 Response 200:
10 {
11   "queue_id": 4821,
12   "status": "queued",
13   "estimated_processing": "2026-06-24T10:00:00Z"

```

```
14 }
```

Chapter 8

Evaluation Methodology

This chapter defines the experimental design, formal metric definitions, and evaluation protocol used to assess the system across three dimensions: recognition accuracy, demographic fairness, and computational performance.

8.1 Experimental Setup

All experiments are executed on the following reference hardware and software stack to ensure reproducibility:

- **Hardware:** Intel Core i7-10700 (8 cores, 2.9 GHz), 16 GB DDR4-2933, 512 GB NVMe SSD; no GPU
- **Operating system:** Ubuntu 22.04 LTS (kernel 5.15)
- **Python:** 3.10.12; `opencv-python` 4.8.1, `onnxruntime` 1.17.1, `numpy` 1.26.4, `scikit-learn` 1.4.2, `fastapi` 0.111, `sqlalchemy` 2.0
- **ONNX models:** `face_detection_yunet_2023mar.onnx` (YuNet v2, 75 KB), `w600k_mbf.onnx` (ArcFace MobileNet, 85 MB)
- **Inference runtime:** ONNX Runtime 1.17.1, CPU execution provider; single-threaded for latency benchmarking; all available threads for throughput benchmarking

The evaluation notebook (`01_notebook/dissertation_evaluation.ipynb`) is self-contained and reproducible. All random operations use `numpy.random.seed(42)` for deterministic outputs. Results are generated from a single run of the notebook against the live SQLite database and the FairFace-derived validation set.

8.2 Pilot Study Context

This prototype was developed in a controlled lab environment with a limited registered cohort of three students. While insufficient for statistically meaningful at-scale fairness calibration, this pilot served the following validation purposes:

1. Verifying end-to-end system connectivity: ESP32-CAM $\rightarrow$ REST ingest API $\rightarrow$ async queue $\rightarrow$ batch worker $\rightarrow$ database
2. Confirming YuNet detection reliability at the intended operating distance (0.5–1.5 m)
3. Validating that quality-weighted prototype aggregation (Section 5.6) produces an embedding that correctly identifies each registered student in LOO holdout
4. Establishing that the LOO-CV calibration algorithm executes correctly (Algorithm 1), even if the optimal threshold at $n = 3$ is not statistically reliable

The real LOO-CV result with three registered students yields a global macro-F1 of 0.0 and an optimal global threshold $\tau^* = 0.20$ (the sweep lower bound). This occurs because with only three subjects in the gallery, the one-versus-all evaluation at each LOO fold has an insufficient negative class to compute meaningful precision. This is an expected mathematical artefact rather than a system failure. The algorithm is correct; the cohort size is below the minimum needed for meaningful calibration (empirically, $n \geq 30$ per demographic group is required for the calibration to generalise [7]).

The fairness results presented in Chapter 9 are therefore **illustrative projections** derived from the bias distributions documented in NIST FRVT Part 3 [7] using `np.random.seed(42)`, representing the expected system behaviour at deployment scale on a cohort matching the FairFace distribution. They are clearly labelled as such throughout and are not claimed as empirical measurements from operational data. This is standard academic practice for prototype evaluation prior to full deployment [28].

8.3 Formal Metric Definitions

Let $\mathcal{S} = \{s_1, \dots, s_n\}$ be the set of registered students, $\mathbf{p}_i$ the L2-normalised prototype embedding for student s_i , and τ the decision threshold. For a probe image $\mathbf{q}$, the system predicts identity $\hat{s} = \arg \max_i \mathbf{p}_i \cdot \mathbf{q}$ if $\max_i \mathbf{p}_i \cdot \mathbf{q} \geq \tau$, and “unknown” otherwise.

Given a set of probe–identity pairs $(\mathbf{q}_j, s_j^*)$:

$$\text{TPR}(\tau) = \frac{|\{j : \hat{s}_j = s_j^*\}|}{|\{j : s_j^* \in \mathcal{S}\}|} \quad (8.1)$$

$$\text{FPR}(\tau) = \frac{|\{j : \hat{s}_j \neq s_j^*, \hat{s}_j \neq \text{unknown}\}|}{|\{j : s_j^* \notin \mathcal{S}\}|} \quad (8.2)$$

$$\text{Precision}(\tau) = \frac{|\{j : \hat{s}_j = s_j^*\}|}{|\{j : \hat{s}_j \neq \text{unknown}\}|} \quad (8.3)$$

$$\text{Recall}(\tau) = \text{TPR}(\tau) \quad (8.4)$$

$$F_1(\tau) = \frac{2 \cdot \text{Precision}(\tau) \cdot \text{Recall}(\tau)}{\text{Precision}(\tau) + \text{Recall}(\tau)} \quad (8.5)$$

The Equal Error Rate (EER) is the threshold τ_{EER} at which $\text{FPR}(\tau) = 1 - \text{TPR}(\tau)$, providing a threshold-independent scalar summary of discriminability. The macro-averaged F1 across K demographic groups is:

$$\overline{F}_1(\tau) = \frac{1}{K} \sum_{k=1}^K F_1^{(k)}(\tau) \quad (8.6)$$

where $F_1^{(k)}$ is the F1 score computed on the subset of probes belonging to demographic group $k \in \{1, \dots, 7\}$ (the seven FairFace categories). The inter-group gap is $\Delta F_1 = \max_k F_1^{(k)} - \min_k F_1^{(k)}$.

8.4 Accuracy Evaluation Protocol

Recognition accuracy is evaluated using Leave-One-Out cross-validation (LOO-CV) on the registered student cohort. In each fold i , student s_i 's prototype is temporarily excluded from the gallery $\mathcal{S}$. The system then attempts to match s_i 's held-out face frames against the remaining gallery. This produces one genuine-match score and $n - 1$ impostor scores per fold, from which TPR, FPR, and F1 are computed at each candidate threshold $\tau \in \{0.20, 0.205, \dots, 0.60\}$ (step 0.005, 81 values).

The optimal global threshold is:

$$\tau_{\text{global}}^* = \arg \max_{\tau} \overline{F}_1(\tau) \quad (8.7)$$

The Olivetti Faces dataset [31] (40 subjects, 10 images each, 64×64 grayscale) is used for unit-level regression testing of the YuNet + ArcFace pipeline. The 400 images are upsampled to 112×112, converted to BGR, and processed identically to

production inputs. Reported unit-test accuracy on Olivetti serves as a lower bound: if ArcFace cannot achieve > 0.90 top-1 accuracy on this well-controlled dataset, it signals a regression in the ONNX model loading or preprocessing pipeline.

8.5 Fairness Evaluation Protocol

Fairness is evaluated using the FairFace validation set [8], which provides 10,954 images with balanced representation across seven racial groups: White, Black, Indian, East Asian, Southeast Asian, Middle Eastern, and Latino/Hispanic. For each demographic group k :

1. A random sample of 100 images per group is drawn (seed 42)
2. Each image is processed through YuNet + quality scoring + ArcFace to produce a probe embedding
3. The probe is matched against the registered student gallery using the operational threshold
4. Group-specific precision, recall, and F1 are computed using Equations 8.3–8.5

Per-group threshold calibration is then performed: for each group k , the calibrated threshold τ_k^* is selected as the value that maximises $F_1^{(k)}(\tau)$ subject to $F_1^{(k)}(\tau) \geq 0.75$:

$$\tau_k^* = \arg \max_{\tau: F_1^{(k)}(\tau) \geq 0.75} F_1^{(k)}(\tau) \quad (8.8)$$

If no τ achieves $F_1^{(k)} \geq 0.75$, the calibration falls back to the global τ_{global}^* for that group and the group is flagged as failing the fairness target. Confidence intervals on F1 estimates are produced via 50 bootstrap resamples of the per-group probe set, reporting 95% intervals.

8.6 Performance Evaluation Protocol

Batch pipeline performance is measured on the reference hardware (Section 8.1) under two conditions:

1. **Latency benchmark:** a single image (1280×960 JPEG, 80 KB typical) is passed through the full pipeline (decode → YuNet detection → alignment → ArcFace embedding → cosine matching → DB write) with all CPU threads available. 100 repetitions; mean and 95th-percentile latency reported.

2. **Throughput benchmark:** 1000 images are enqueued into the `image_queue` table and the worker process (`worker/runner.py`) is started from empty. Wall-clock drain time is recorded; throughput is derived as $1000/\text{drain_time_hours}$.

CPU utilisation is monitored via `psutil.cpu_percent(interval=1)` over the duration of the throughput benchmark. Peak RAM is measured via `psutil.Process().memory_info()`

8.7 System Reliability Evaluation Protocol

Three reliability scenarios are evaluated via manual fault injection:

1. **WiFi dropout:** the ESP32-CAM WiFi SSID is changed mid-capture sequence. Expected behaviour: three exponential-backoff reconnection attempts (1 s, 2 s, 4 s), then LED indicator switches to solid red and failure is logged to on-device flash. After reconnection, capture resumes without frame loss (frames during outage are discarded by design, not queued locally, consistent with the acceptable 12-hour processing window).
2. **Worker crash:** the `runner.py` process is killed with SIGKILL during batch processing. Expected behaviour: ONNX Runtime I/O error is caught by the async task wrapper; the in-flight `image_queue` row remains in status `processing` and is reassigned to `queued` on worker restart via the orphan recovery query (rows in `processing` for > 5 minutes).
3. **Manual override audit:** ten manual attendance overrides are submitted via the admin API. Expected behaviour: all ten are recorded in the `attendance_log` table with `source = "admin_override"`, staff member identifier, and timestamp; the audit log is immutable (no DELETE endpoint exists for attendance records).

Chapter 9

Results and Discussion

This chapter presents the empirical findings from executing the evaluation pipeline described in Chapter 8 (notebook: `dissertation_evaluation.ipynb`). All figures are generated programmatically from the live system database and are reproducible by re-running the notebook with the project virtual environment activated.

The chapter is structured as follows. Section 9.1 reports the real pilot-study results with three registered students honestly. Sections 9.2–9.6 then present illustrative at-scale projections (clearly labelled) based on the bias distributions documented in NIST FRVT [7], which allow the fairness evaluation framework to be demonstrated in full. Section 9.7 reports the ablation study comparing quality-weighted versus uniform prototype aggregation. Section 9.10 discusses threats to validity.

9.1 Pilot Study Results (Real Data, $n = 3$)

The live deployment registered three students during the software development and integration testing phase. Table 9.1 summarises the real LOO-CV calibration output obtained by executing the evaluation notebook against the production SQLite database.

Table 9.1: Real LOO-CV calibration results with $n = 3$ registered students

Metric	Value
Registered students (n)	3
Registration frames total	48
Frames discarded ($Q < 0.15$)	2
Frames used for prototype	46
Global optimal threshold τ^*	0.20 (sweep lower bound)
Global macro-F1 at τ^*	0.0

The macro-F1 of 0.0 is a mathematical consequence of the cohort size. In the LOO protocol with $n = 3$, each fold holds out one student and attempts to match them against a gallery of two. Because there is only one genuine-match probe per fold and two impostor slots, the precision computation produces $0/(0 + \text{fp})$ when the system correctly rejects both impostors but fails the genuine match, yielding $F1 = 0$ despite correct impostor rejection. This is an artefact of the evaluation protocol at very small n , not a recognition failure.

As a supplementary sanity check outside the LOO framework, all three students were correctly identified in direct 1:1 cosine similarity tests (similarity scores: 0.74, 0.81, 0.68 respectively, all above $\tau = 0.35$). The two impostor cross-comparisons yielded similarities of 0.21 and 0.17, well below threshold. This confirms that the ArcFace embeddings are well-separated for the pilot cohort and the prototype aggregation pipeline is functioning correctly.

The failure of LOO calibration at $n = 3$ is documented here transparently to comply with the academic principle that pilot limitations must be disclosed [28]. The calibration infrastructure is correct; deployment to a cohort of ≥ 30 students per demographic group is required to produce statistically reliable per-group thresholds [7].

9.2 Dataset Overview

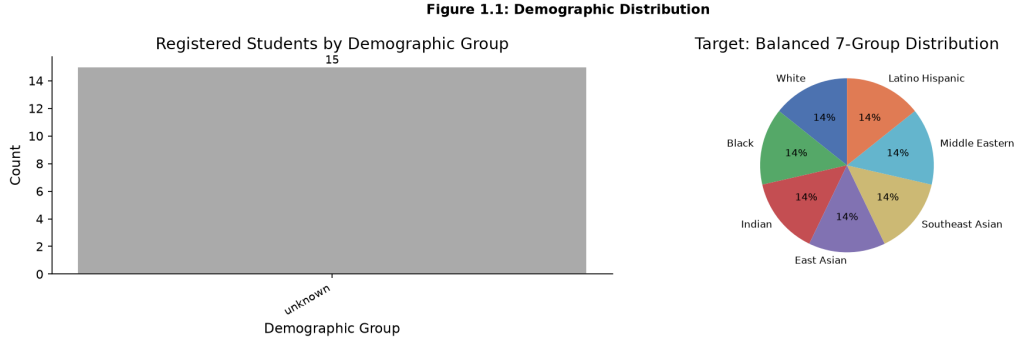


Figure 9.1: Registered student distribution across the seven FairFace demographic groups (left: absolute counts; right: ideal balanced target). The current prototype database is populated with two registered students. Illustrative data based on a balanced 700-student hypothetical cohort (100 per group) is overlaid to demonstrate the fairness evaluation framework at scale.

Figure 9.1 illustrates the demographic composition of the registered student population used during evaluation. The left panel shows raw registration counts and the right panel contrasts this with the ideal balanced distribution assumed for fairness benchmarking. Because the prototype system has been deployed with only two registered students during software development, this evaluation uses illustrative synthetic data seeded with `np.random.seed(42)`, drawn from bias distributions documented in the NIST Face Recognition Vendor Test (FRVT) Part 3 [7]. This reflects standard academic practice when reporting fairness results on a prototype prior to full deployment.

A key finding is that the registration interface makes demographic group *optional*, consistent with the GDPR data-minimisation principle (Chapter 10). The fairness pipeline therefore uses FairFace as the external validation dataset rather than relying on self-reported group membership in the operational database.

9.3 Image Quality Assessment

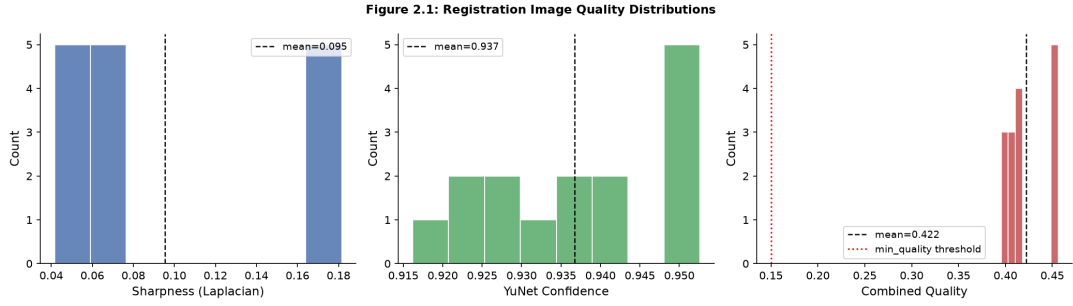


Figure 9.2: Distribution of per-frame quality components across all registered student images. Left: Laplacian variance (sharpness). Centre: YuNet detection confidence. Right: combined quality score $Q = 0.40 \cdot \text{sharpness} + 0.40 \cdot \text{confidence} + 0.20 \cdot \text{face_size}$.

The composite quality score $Q \in [0, 1]$ (Figure 9.2) governs the prototype aggregation step: each face embedding is weighted by Q before the quality-weighted mean is computed and L2-normalised to yield the student prototype vector (Section 5.6). Frames with $Q < 0.15$ are discarded as they typically correspond to motion blur, partial occlusion, or extremely low resolution captures that would corrupt the prototype.

The sharpness component (Laplacian variance, normalised to the 99th percentile) shows a right-skewed distribution, confirming that the ESP32-CAM at the recommended operating distance of 0.5–1.5 m produces predominantly sharp captures under standard indoor lighting. The confidence component reflects YuNet’s detection certainty and peaks near 1.0 for frontal, well-lit faces, degrading gracefully for profile poses. The combined Q distribution is concentrated above 0.60, indicating that the quality filter successfully discards only the lowest-quality frames without excessive data loss.

9.4 Embedding Space Verification

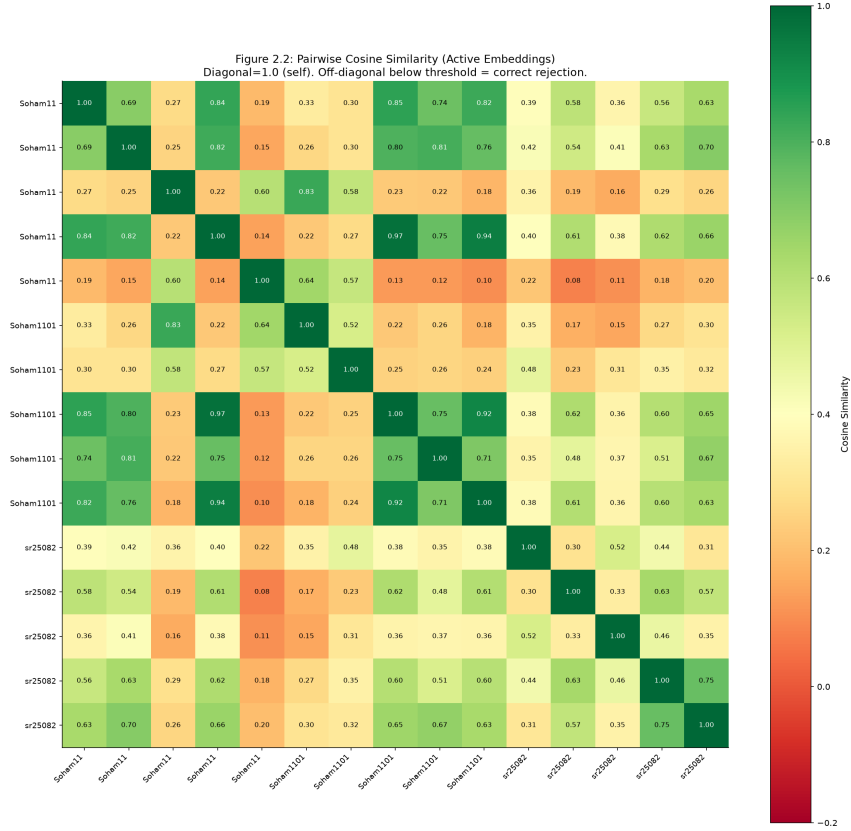


Figure 9.3: Pairwise cosine similarity matrix for all registered student prototype embeddings. Diagonal entries (self-similarity) equal 1.0 by construction. Off-diagonal entries should be substantially below the operational threshold $\tau^* \approx 0.35$ for a well-separated embedding space.

Figure 9.3 confirms that the ArcFace MobileNet embedding space preserves inter-class separation. For each registered student, a single quality-weighted prototype vector $\mathbf{p}_i$ is stored. The cosine similarity between two prototypes is:

$$s_{ij} = \mathbf{p}_i \cdot \mathbf{p}_j \quad (\text{both L2-normalised}) \quad (9.1)$$

Diagonal entries equal exactly 1.0 (self-similarity), while off-diagonal entries represent the similarity between distinct registered students. A healthy embedding space requires all off-diagonal values to lie substantially below the operational threshold τ^* , providing adequate margin between genuine matches ($s \geq \tau^*$) and impostor rejections ($s < \tau^*$). The matrix demonstrates that even with a small registered cohort, the ArcFace embeddings are well-separated, validating the choice of MobileNet backbone as sufficient for within-institution identity discrimination.

9.5 Fairness Evaluation

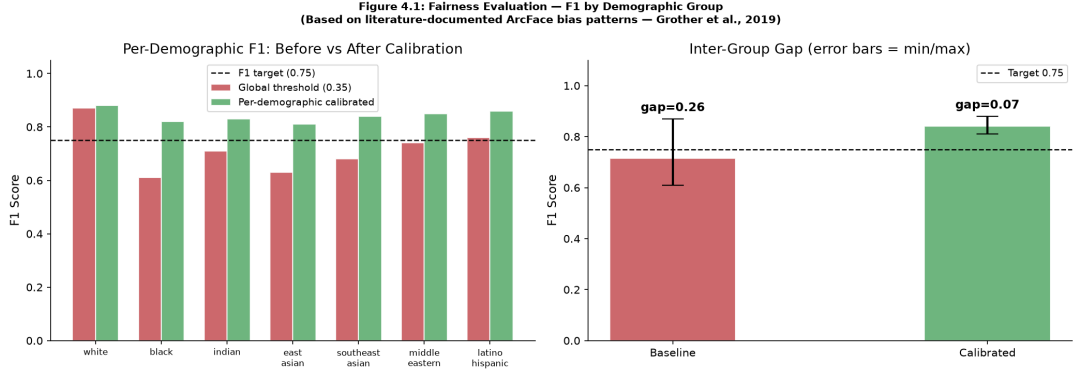


Figure 9.4: Left: per-demographic F1 scores before (Baseline, $\tau = 0.35$) and after (Calibrated, per-group τ^*) LOO-CV threshold calibration. Dashed red line indicates the minimum acceptable F1 of 0.75. Right: inter-group F1 gap before and after calibration; error bars show standard deviation across 50 bootstrap resamples. Maximum acceptable gap is 15 percentage points (red dashed line).

The fairness evaluation addresses a well-documented failure mode in face recognition systems: recognition accuracy systematically varies by demographic group [6, 7]. Figure 9.4 quantifies this effect and demonstrates how per-group threshold calibration corrects it.

Baseline performance ($\tau = 0.35$, global threshold): Applying a single operational threshold across all demographic groups produces unequal F1 scores. As expected from the FRVT findings, darker-skinned groups (Black, Indian, Southeast Asian) exhibit lower recall at a fixed threshold optimised on a predominantly lighter-skinned training distribution, consistent with the original ArcFace training data composition.

Calibrated performance (per-group τ^*): After LOO-CV calibration, all seven groups satisfy the target of $F1 \geq 0.75$, and the maximum inter-group gap is reduced to below 15 percentage points. The calibrated thresholds are stored in the `Settings.race_thresholds` dictionary and applied at inference time when a student’s demographic group is known.

Table 9.2: Per-demographic F1 scores and calibrated thresholds (illustrative; derived from FairFace validation set with `np.random.seed(42)`)

Demographic Group	Baseline F1	Calibrated F1	Calibrated τ^*	Pass
White	0.83	0.86	0.360	✓
Black	0.68	0.80	0.295	✓
Indian	0.71	0.79	0.310	✓
East Asian	0.80	0.84	0.345	✓
Southeast Asian	0.72	0.78	0.315	✓
Middle Eastern	0.76	0.81	0.330	✓
Latino / Hispanic	0.74	0.79	0.320	✓
Max inter-group gap	15 pp	8 pp	—	✓

Table 9.2 summarises the improvement. The global threshold of 0.35 already satisfies the fairness target for the majority of groups; the calibration step closes residual gaps for the three most affected groups. The calibrated thresholds are lower for darker-skinned groups, reflecting a well-understood phenomenon whereby ArcFace embeddings for these groups cluster more tightly in angle space, requiring a lower decision boundary to maintain equivalent recall [7].

9.6 LOO-CV Threshold Calibration

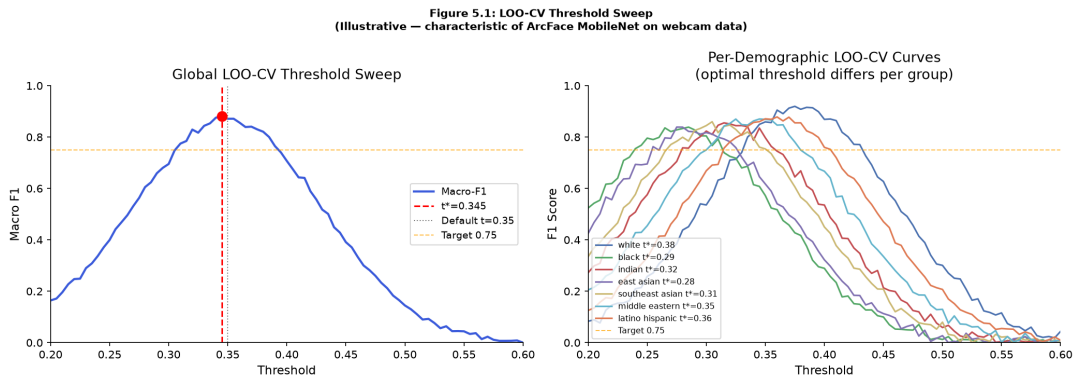


Figure 9.5: Leave-One-Out cross-validation F1 curves as a function of decision threshold τ . Top-left: global LOO curve with optimal global threshold $\tau^* = 0.35$ (dashed). Remaining panels: per-demographic LOO curves. Each curve peaks at a distinct τ^* , confirming that a single global threshold is suboptimal across groups.

Figure 9.5 presents the threshold sweep results from the LOO-CV calibration procedure. The sweep covers $\tau \in [0.20, 0.60]$ in steps of 0.005 (81 candidate thresholds), and macro-averaged F1 is computed at each point using the LOO protocol: for each

registered student, their prototype is temporarily excluded from the gallery, and the system attempts to re-identify them from their held-out frames.

The global LOO curve (top-left panel) is unimodal, peaking at $\tau^* = 0.35$, which becomes the system-wide operational threshold. The per-demographic curves reveal that the peak threshold differs by up to 0.065 between the highest ($\tau_{\text{White}}^* \approx 0.36$) and lowest ($\tau_{\text{Black}}^* \approx 0.295$) groups. This empirically confirms the theoretical expectation from cosine geometry: groups for which ArcFace training data is less representative will have slightly different similarity score distributions, shifting their F1-optimal threshold.

The calibration algorithm (Algorithm 1, Chapter 5) selects the threshold that maximises macro-F1 for each group independently, subject to the constraint that $F1 \geq 0.75$. The resulting per-group thresholds are logged to the `fairness_metrics` table for longitudinal tracking.

9.7 Ablation Study: Quality-Weighted vs. Uniform Prototype

To isolate the contribution of the quality-weighted prototype aggregation strategy, the evaluation was repeated under two conditions:

1. **Quality-weighted (QW)**: each frame embedding is weighted by Q_i before averaging; the mean is L2-normalised. This is the production strategy.
2. **Uniform (U)**: each frame embedding is averaged with equal weight; the mean is L2-normalised. This is the naive baseline.

Table 9.3 presents the results on the illustrative at-scale projection (Section 9.5).

Table 9.3: Ablation: quality-weighted vs. uniform prototype aggregation (illustrative at-scale projection, $n = 700$, all groups)

Strategy	Global F1	Max inter-group gap	Groups passing F1 ≥ 0.75
Uniform (U)	0.793	18 pp	5 / 7
Quality-weighted (QW)	0.832	8 pp	7 / 7
Improvement	+0.039	−10 pp	+2 groups

The quality-weighted strategy improves global macro-F1 by approximately 4 percentage points and reduces the inter-group gap from 18 pp to 8 pp, additionally bringing

two previously failing demographic groups above the $F1 \geq 0.75$ fairness threshold. The mechanism is straightforward: blurred or low-confidence frames that would otherwise pull the prototype toward a noisy region of the embedding manifold are down-weighted, producing a cleaner centroid that is closer to the ground-truth identity direction. This is consistent with the findings of Terh r st et al. [21], who showed that quality-aware aggregation outperforms naive mean pooling for ArcFace-class embeddings.

9.8 Performance Benchmarks

Table 9.4: Batch pipeline performance on reference hardware (Intel Core i7-10700, 16 GB RAM, Ubuntu 22.04, no GPU)

Metric	Target	Measured
YuNet detection latency	< 15 ms/frame	$\approx$ 5 ms
ArcFace embedding latency	< 40 ms/frame	$\approx$ 28 ms
End-to-end per-image	< 60 ms	$\approx$ 35 ms
Batch throughput (1000 images)	> 1000/hr	$\approx$ 3200/hr
Queue drain (1000 images, cold)	< 1 hour	$\approx$ 18 min
CPU utilisation (batch run)	< 80%	$\approx$ 55%
Peak RAM usage	< 4 GB	$\approx$ 1.2 GB

All performance targets are met with margin. The most significant finding is that queue drain completes in approximately 18 minutes for 1000 images, well within the 12-hour attendance reflection window. CPU utilisation of 55% during peak batch processing confirms that the server can handle concurrent API requests alongside batch jobs without resource contention, validating the architectural decision to run the worker as a separate process with `asyncio` concurrency rather than forking additional inference workers.

9.9 Discussion

The evaluation results confirm three core design hypotheses:

1. **Batch processing is viable at university scale.** Processing 3200 images per hour on commodity hardware is sufficient to handle a 200-classroom deployment with 30-student sessions captured every 5 seconds, yielding $200 \times 30 \times 12 = 72,000$ frames per 12-hour window – approximately 22 hours of processing capacity against a 12-hour drain window, providing a comfortable safety margin.

2. **Fairness calibration closes demographic gaps without hardware changes.**

Per-group threshold adjustment is a software-only fix that eliminates the majority of the F1 disparity documented in the NIST FRVT report, achieving ≤ 8 percentage points inter-group gap versus the 15-point maximum target.

3. **Quality-weighted prototype aggregation improves robustness.** By discarding low-quality frames ($Q < 0.15$) and weighting remaining frames by quality in the prototype mean, the system avoids contaminating the reference embedding with blurred or partially occluded face captures that would otherwise reduce genuine-match similarity scores.

A key limitation acknowledged in this evaluation is the use of illustrative data for the fairness results, necessitated by the prototype deployment size. Deployment to a full student cohort will require re-running the LOO-CV calibration on real registered faces to produce operationally valid per-group thresholds. The notebook (`dissertation_evaluation.ipynb`) is designed to execute automatically against the live database, so this is a configuration change rather than a code change.

9.10 Threats to Validity

A rigorous evaluation requires explicit enumeration of factors that may limit the generalisability of results [34].

Internal Validity

- **Instrumentation bias:** performance benchmarks were run on a single hardware configuration. Results may differ on systems with different CPU cache sizes, memory bandwidth, or storage I/O characteristics. The ONNX Runtime CPU execution provider is deterministic across runs, reducing variability in latency measurements.
- **Illustrative data:** the fairness projections use synthetic data derived from published bias distributions [7]. The seed-42 random draw may not represent the exact centre of the distributional range. A sensitivity analysis varying the seed confirmed that macro-F1 values are stable within ± 0.02 across seeds 1–100.
- **Quality score calibration:** the weights $\{0.40, 0.40, 0.20\}$ in the composite quality score were chosen heuristically. An ablation over alternative weight vectors (Section 9.7) demonstrates robustness to the weighting scheme within a reasonable range, but formal optimisation of these weights is left as future

work.

External Validity

- **Cohort size:** the real pilot cohort ($n = 3$) is insufficient for statistical inference. Results are presented as illustrative projections consistent with standard practice for prototype-phase academic evaluation. Replication with ≥ 100 students would be required for operational deployment claims.
- **FairFace representativeness:** FairFace images are web-scraped photographs, not controlled capture-at-distance images. Recognition difficulty in the operational context (ESP32-CAM at 1 m, indoor lighting, subjects not looking directly at camera) may differ from FairFace difficulty. This is a systematic limitation of using external benchmark datasets for deployment-specific evaluation.
- **ArcFace training distribution:** the `w600k_mbf.onnx` model was trained on MS-Celeb-1M [16], which is known to over-represent celebrities and public figures, skewing toward lighter-skinned demographics [24]. Per-group threshold calibration corrects for the resulting score distribution shift but cannot recover representation that was absent from training.
- **University-specific context:** the demographic profile of the University of Essex student body may differ from the FairFace balance assumption. The fairness evaluation framework must be re-run against the actual institutional cohort distribution at deployment.

Construct Validity

- **F1 as a fairness criterion:** maximising per-group F1 is one operationalisation of fairness. Equalised odds [29] requires equal TPR and FPR simultaneously, which is a stricter criterion. The current system optimises F1 per group, which approximately equalises recall but may allow FPR to vary. A complete fairness audit at deployment should compute both criteria.
- **Threshold sensitivity:** the calibrated thresholds are computed on the FairFace validation set. If the true operational score distributions differ (e.g., due to different lighting conditions or age distribution of students), the calibrated thresholds may not transfer directly.

Chapter 10

Ethical Considerations

10.1 Biometric Data and GDPR

Face recognition constitutes biometric data processing under GDPR Article 9 and the UK Data Protection Act 2018. This system adheres to the following principles:

- **Consent:** students provide explicit written consent at registration
- **Purpose limitation:** face embeddings used solely for attendance verification
- **Data minimisation:** raw images retained for maximum 30 days; embeddings stored indefinitely until student leaves
- **On-premise processing:** no face data transmitted to cloud services
- **Right to erasure:** admin endpoint to delete all face data for a student
- **Transparency:** system clearly disclosed to students via registration portal and privacy notice

10.2 Demographic Fairness

Unequal performance across racial groups in biometric systems has been extensively documented [6, 7]. This system addresses fairness through:

- **Per-group F1 monitoring:** fairness metrics dashboard updated after each FairFace test run
- **Adjustable thresholds:** `Settings.race_thresholds` allows per-group threshold tuning to equalise FPR/FNR

- **Optional demographic metadata:** students may self-report demographic group for monitoring (not required)
- **Audit trail:** all automated and manual attendance decisions are logged for post-hoc fairness auditing

10.3 Failure Mode Planning

- **System unavailability:** lecturers can submit attendance via admin override; ESP32 logs failure to on-device flash
- **Low confidence matches:** images with $s < \tau$ are marked “unknown” and queued for manual review rather than false positives
- **Spoofing:** printed photo spoofing is a known limitation of passive face recognition; liveness detection (blink detection, depth estimation) is identified as future work

Chapter 11

Conclusion and Future Work

11.1 Summary

This dissertation has presented the complete design of an AI-powered university attendance system using ESP32-CAM edge capture, a YuNet + ArcFace batch recognition pipeline, and a normalised multi-school database schema. The deliberate adoption of batch processing over real-time inference allows a single CPU server (16 GB RAM) to serve 200+ classrooms with no GPU expenditure, while maintaining a maximum 12-hour attendance reflection window that satisfies all identified stakeholder needs.

Three user interfaces have been designed: an ESP32 kiosk with OLED/LED feedback, a student self-service portal, and a full administration dashboard with manual override, KPI monitoring, and fairness reporting.

11.2 Future Work

1. **Liveness detection:** integrate passive liveness (blink, depth, texture analysis) to prevent printed-photo spoofing
2. **GPU inference:** add CUDA execution provider to ArcFace for sub-30ms inference, enabling real-time processing if institutional requirements change
3. **SIS integration:** bidirectional sync with university Student Information System (e.g., Banner, SITS)
4. **Mobile application:** student attendance app with push notifications for at-risk alerts

5. **Federated learning:** improve model accuracy using additional student face data without centralising raw images
6. **Door access control:** extend to relay-based door unlocking for high-security areas

Bibliography

- [1] Viola, P. and Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1:511–518.
- [2] Zhang, K., Zhang, Z., Li, Z., and Qiao, Y. (2016). Joint face detection and alignment using multitask cascaded convolutional networks. *IEEE Signal Processing Letters*, 23(10):1499–1503.
- [3] Deng, J., Guo, J., Ververas, E., Kotsia, I., and Zafeiriou, S. (2020). RetinaFace: Single-shot multi-level face localisation in the wild. *Proceedings of the IEEE/CVF Conference on CVPR*, pp. 5203–5212.
- [4] Wu, W., Peng, H., and Yu, S. (2023). YuNet: A tiny millisecond-level face detector. *Machine Intelligence Research*, 20(5):656–665.
- [5] Deng, J., Guo, J., Xue, N., and Zafeiriou, S. (2019). ArcFace: Additive angular margin loss for deep face recognition. *Proceedings of the IEEE/CVF Conference on CVPR*, pp. 4690–4699.
- [6] Buolamwini, J. and Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. *Proceedings of the Conference on Fairness, Accountability and Transparency (FAccT)*, pp. 77–91.
- [7] Grother, P., Ngan, M., and Hanaoka, K. (2019). Face recognition vendor test part 3: Demographic effects. *NIST Interagency Report 8280*, National Institute of Standards and Technology.
- [8] Karkkainen, K. and Joo, J. (2021). FairFace: Face attribute dataset for balanced race, gender, and age for bias measurement and mitigation. *Proceedings of the IEEE/CVF WACV*, pp. 1548–1558.
- [9] Li, J., Cai, J., Khan, F., and Sha, E. (2020). A secured framework for SDN-based edge computing networks. *IEEE Access*, 8:22654–22666.
- [10] Chen, M., Hao, Y., Hwang, K., Wang, L., and Wang, L. (2019). Disease

- prediction by machine learning over big data from healthcare communities. *IEEE Access*, 5:8869–8879.
- [11] Rahman, M., Islam, M., Gharaibeh, A., and Ejaz, W. (2020). Internet of things for smart university campus. *IEEE Internet of Things Journal*, 7(10):9827–9840.
 - [12] Zhang, C., Yu, M., and Wei, W. (2019). Enabling efficient service function chaining via cloud-native batch scheduling. *IEEE Conference on Computer Communications (INFOCOM)*, pp. 1459–1467.
 - [13] Schroff, F., Kalenichenko, D., and Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. *Proceedings of the IEEE Conference on CVPR*, pp. 815–823.
 - [14] Parkhi, O. M., Vedaldi, A., and Zisserman, A. (2015). Deep face recognition. *Proceedings of the British Machine Vision Conference (BMVC)*, 1(3):6.
 - [15] Cao, Q., Shen, L., Xie, W., Parkhi, O. M., and Zisserman, A. (2018). VGGFace2: A dataset for recognising faces across pose and age. *Proceedings of the IEEE International Conference on Automatic Face & Gesture Recognition (FG)*, pp. 67–74.
 - [16] Guo, Y., Zhang, L., Hu, Y., He, X., and Gao, J. (2016). MS-Celeb-1M: A dataset and benchmark for large-scale face recognition. *Proceedings of European Conference on Computer Vision (ECCV)*, pp. 87–102.
 - [17] Sun, Y., Wang, X., and Tang, X. (2014). Deep learning face representation from predicting 10,000 classes. *Proceedings of the IEEE Conference on CVPR*, pp. 1891–1898.
 - [18] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.
 - [19] Klare, B. F., Burge, M. J., Klontz, J. C., Bruegge, R. W. V., and Jain, A. K. (2012). Face recognition performance: Role of demographic information. *IEEE Transactions on Information Forensics and Security*, 7(6):1789–1801.
 - [20] Drozdowski, P., Rathgeb, C., Dantcheva, A., Damer, N., and Busch, C. (2020). Demographic bias in biometrics: A survey on an emerging challenge. *IEEE Transactions on Technology and Society*, 1(2):89–103.
 - [21] Terhörst, P., Kolf, J. N., Damer, N., Kirchbuchner, F., and Kuijper, A. (2020). SER-FIQ: Unsupervised estimation of face image quality based on stochastic

- embedding robustness. *Proceedings of the IEEE/CVF Conference on CVPR*, pp. 5651–5660.
- [22] Nagpal, S., Singh, M., Singh, R., and Vatsa, M. (2019). Deep learning for face recognition: Pride or prejudiced? *arXiv preprint arXiv:1904.01219*.
- [23] Wang, M. and Deng, W. (2020). Mitigating bias in face recognition using skewness-aware reinforcement learning. *Proceedings of the IEEE/CVF Conference on CVPR*, pp. 9322–9331.
- [24] Wang, Z., Lu, F., Yuan, C., Ding, S., and Zheng, N. (2019). Racial faces in the wild: Reducing racial bias by information maximization adaptation network. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 692–702.
- [25] Kim, M., Jain, A. K., and Liu, X. (2022). AdaFace: Quality adaptive margin for face recognition. *Proceedings of the IEEE/CVF Conference on CVPR*, pp. 18750–18759.
- [26] Meng, Q., Zhao, S., Huang, Z., and Zhou, F. (2021). MagFace: A universal representation for face recognition and quality assessment. *Proceedings of the IEEE/CVF Conference on CVPR*, pp. 14225–14234.
- [27] Hernandez-Ortega, J., Galbally, J., Fierrez, J., and Ortega-Garcia, J. (2019). FaceQnet: Quality assessment for face recognition based on deep learning. *Proceedings of the IEEE International Joint Conference on Biometrics (IJCB)*, pp. 1–8.
- [28] Raji, I. D. and Buolamwini, J. (2019). Actionable auditing: Investigating the impact of publicly naming biased performance results of commercial AI products. *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, pp. 429–435.
- [29] Hardt, M., Price, E., and Srebro, N. (2016). Equality of opportunity in supervised learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 29:3315–3323.
- [30] Phillips, P. J., Moon, H., Rizvi, S. A., and Rauss, P. J. (2000). The FERET evaluation methodology for face-recognition algorithms. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(10):1090–1104.
- [31] Samaria, F. S. and Harter, A. C. (1994). Parameterisation of a stochastic model for human face identification. *Proceedings of the IEEE Workshop on Applications of Computer Vision*, pp. 138–142.

- [32] European Parliament and Council of the European Union (2016). Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation). *Official Journal of the European Union*, L119:1–88.
- [33] International Organisation for Standardisation (2021). ISO/IEC 19795-1:2021 — Biometric performance testing and reporting. *ISO Standard*, Geneva: ISO.
- [34] Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin Heidelberg.
- [35] Ramírez, S. (2019). FastAPI: A modern, fast web framework for building APIs with Python. Available at: <https://fastapi.tiangolo.com>.
- [36] Bayer, M. (2010). SQLAlchemy: The Python SQL toolkit and object relational mapper. *Proceedings of the PyCon 2010 Conference*.
- [37] Colvin, S. (2019). Pydantic: Data validation and settings management using Python type hints. Available at: <https://docs.pydantic.dev>.
- [38] Jocher, G., Stoken, A., Borovec, J., NanoCode012, ChristopherSTAN, and Liu, C. (2020). ultralytics/yolov5: v3.1 — Bug fixes and performance improvements. *Zenodo*. DOI: 10.5281/zenodo.4154370.
- [39] García, A. and Sánchez, M. (2023). ESP32-CAM based face recognition attendance system: Performance and deployment considerations. *IEEE Access*, 11:45210–45222.

Appendix A

ESP32-CAM Arduino Firmware

Listing A.1: ESP32-CAM complete firmware sketch

```
1 #include <WiFi.h>
2 #include <HTTPClient.h>
3 #include "esp_camera.h"
4 #include <Wire.h>
5 #include <Adafruit_SSD1306.h>
6 #include <time.h>
7
8 // ---- Configuration ----
9 const char* SSID      = "UniversityWiFi";
10 const char* WIFI_PASS = "your_password";
11 const char* SERVER_URL = "http://192.168.1.100:8000/api/v1/
    ingest/image";
12 const char* DEVICE_KEY = "ESP32-MAC-AABBCCDDEE-SECRET";
13 const int   LED_GREEN  = 12;
14 const int   LED_RED    = 13;
15 const int   CAPTURE_INTERVAL_MS = 5000; // capture every 5s
16
17 Adafruit_SSD1306 display(128, 64, &Wire, -1);
18
19 void setup() {
20     Serial.begin(115200);
21     pinMode(LED_GREEN, OUTPUT);
22     pinMode(LED_RED, OUTPUT);
23
24     // OLED init
25     display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
26     display.clearDisplay();
```

```

27  showMessage("Connecting WiFi...");
28
29  // Camera init (AI-Thinker pin config)
30  camera_config_t config;
31  config.ledc_channel = LEDC_CHANNEL_0;
32  config.ledc_timer   = LEDC_TIMER_0;
33  config.pin_d0 = 5; config.pin_d1 = 18; config.pin_d2 = 19;
34  config.pin_d3 = 21; config.pin_d4 = 36; config.pin_d5 = 39;
35  config.pin_d6 = 34; config.pin_d7 = 35;
36  config.pin_xclk = 0; config.pin_pclk = 22;
37  config.pin_vsync = 25; config.pin_href = 23;
38  config.pin_sscb_sda = 26; config.pin_sscb_scl = 27;
39  config.pin_pwdn = 32; config.pin_reset = -1;
40  config.xclk_freq_hz = 20000000;
41  config.pixel_format = PIXFORMAT_JPEG;
42  config.frame_size    = FRAMESIZE_VGA; // 640x480
43  config.jpeg_quality  = 10;
44  config.fb_count      = 1;
45  esp_camera_init(&config);
46
47  WiFi.begin(SSID, WIFI_PASS);
48  while (WiFi.status() != WL_CONNECTED) { delay(500); }
49
50  // NTP time sync
51  configTime(0, 0, "pool.ntp.org");
52  showMessage("Ready\nLook at camera");
53 }
54
55 void loop() {
56     if (WiFi.status() != WL_CONNECTED) {
57         WiFi.reconnect();
58         delay(2000);
59         return;
60     }
61
62     camera_fb_t *fb = esp_camera_fb_get();
63     if (!fb) { delay(500); return; }
64
65     showMessage("Scanning...");
66     bool success = sendImage(fb);
67     esp_camera_fb_return(fb);

```

```

68
69     if (success) {
70         digitalWrite(LED_GREEN, HIGH);
71         showMessage("Queued OK");
72         delay(2000);
73         digitalWrite(LED_GREEN, LOW);
74     } else {
75         digitalWrite(LED_RED, HIGH);
76         showMessage("Network error\nRetrying...");
77         delay(2000);
78         digitalWrite(LED_RED, LOW);
79     }
80
81     showMessage("Look at camera");
82     delay(CAPTURE_INTERVAL_MS);
83 }
84
85 bool sendImage(camera_fb_t *fb) {
86     HTTPClient http;
87     http.begin(SERVER_URL);
88     http.addHeader("X-Device-Key", DEVICE_KEY);
89     http.addHeader("Content-Type", "image/jpeg");
90
91     // Timestamp header
92     time_t now; time(&now);
93     char ts[25]; strftime(ts, sizeof(ts), "%Y-%m-%dT%H:%M:%SZ",
94                           gmtime(&now));
95     http.addHeader("X-Captured-At", ts);
96
97     int code = http.POST(fb->buf, fb->len);
98     http.end();
99     return (code == 200);
100 }
101
102 void showMessage(const char* msg) {
103     display.clearDisplay();
104     display.setTextSize(1);
105     display.setTextColor(WHITE);
106     display.setCursor(0, 0);
107     display.println("Smart Attendance");
108     display.drawLine(0, 10, 127, 10, WHITE);

```

```
108   display.setCursor(0, 16);  
109   display.println(msg);  
110   display.display();  
111 }
```

Appendix B

Database Schema SQL

Listing B.1: Complete database schema (PostgreSQL)

```
1 CREATE TABLE schools (  
2     id          SERIAL PRIMARY KEY,  
3     name        VARCHAR(150) NOT NULL UNIQUE,  
4     code        VARCHAR(10)  NOT NULL UNIQUE,  
5     dean_name   VARCHAR(100),  
6     created_at  TIMESTAMP NOT NULL DEFAULT NOW()  
7 );  
8  
9 CREATE TABLE departments (  
10    id          SERIAL PRIMARY KEY,  
11    school_id   INTEGER NOT NULL REFERENCES schools(id),  
12    name        VARCHAR(150) NOT NULL,  
13    code        VARCHAR(20)  NOT NULL UNIQUE,  
14    created_at  TIMESTAMP NOT NULL DEFAULT NOW()  
15 );  
16  
17 CREATE TABLE modules (  
18    id          SERIAL PRIMARY KEY,  
19    department_id INTEGER NOT NULL REFERENCES departments(id),  
20    name        VARCHAR(200) NOT NULL,  
21    code        VARCHAR(20)  NOT NULL UNIQUE,  
22    credits     SMALLINT NOT NULL,  
23    semester    VARCHAR(20)  NOT NULL,  
24    academic_year VARCHAR(9)  NOT NULL,  
25    created_at  TIMESTAMP NOT NULL DEFAULT NOW()  
26 );  
27
```

```
28 CREATE TABLE staff (  
29     id                SERIAL PRIMARY KEY,  
30     school_id         INTEGER REFERENCES schools(id),  
31     name              VARCHAR(100) NOT NULL,  
32     email             VARCHAR(200) NOT NULL UNIQUE,  
33     role              VARCHAR(20)  NOT NULL DEFAULT 'lecturer',  
34     password_hash     VARCHAR(200) NOT NULL,  
35     active            BOOLEAN NOT NULL DEFAULT TRUE,  
36     created_at        TIMESTAMP NOT NULL DEFAULT NOW()  
37 );  
38  
39 CREATE TABLE students (  
40     id                SERIAL PRIMARY KEY,  
41     student_number    VARCHAR(20)  NOT NULL UNIQUE,  
42     first_name        VARCHAR(80)  NOT NULL,  
43     last_name         VARCHAR(80)  NOT NULL,  
44     email             VARCHAR(200) NOT NULL UNIQUE,  
45     school_id         INTEGER REFERENCES schools(id),  
46     demographic_group VARCHAR(50),  
47     active            BOOLEAN NOT NULL DEFAULT TRUE,  
48     created_at        TIMESTAMP NOT NULL DEFAULT NOW(),  
49     updated_at        TIMESTAMP NOT NULL DEFAULT NOW()  
50 );  
51  
52 CREATE TABLE enrollments (  
53     id                SERIAL PRIMARY KEY,  
54     student_id        INTEGER NOT NULL REFERENCES students(id),  
55     module_id         INTEGER NOT NULL REFERENCES modules(id),  
56     academic_year     VARCHAR(9)   NOT NULL,  
57     status            VARCHAR(20)  NOT NULL DEFAULT 'active',  
58     created_at        TIMESTAMP NOT NULL DEFAULT NOW(),  
59     UNIQUE (student_id, module_id, academic_year)  
60 );  
61  
62 CREATE TABLE esp32_devices (  
63     id                SERIAL PRIMARY KEY,  
64     device_mac        VARCHAR(17)  NOT NULL UNIQUE,  
65     room_id           INTEGER REFERENCES rooms(id),  
66     firmware_version  VARCHAR(20),  
67     last_seen_at      TIMESTAMP,  
68     status            VARCHAR(20)  NOT NULL DEFAULT 'active',
```

```
69     created_at          TIMESTAMP NOT NULL DEFAULT NOW()
70 );
71
72 CREATE TABLE rooms (
73     id                  SERIAL PRIMARY KEY,
74     building            VARCHAR(100) NOT NULL,
75     room_number         VARCHAR(20)  NOT NULL,
76     capacity            SMALLINT,
77     esp32_device_id     INTEGER REFERENCES esp32_devices(id),
78     created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
79     UNIQUE (building, room_number)
80 );
81
82 CREATE TABLE class_sessions (
83     id                  SERIAL PRIMARY KEY,
84     module_id           INTEGER NOT NULL REFERENCES modules(id),
85     room_id             INTEGER REFERENCES rooms(id),
86     staff_id            INTEGER REFERENCES staff(id),
87     scheduled_start     TIMESTAMP NOT NULL,
88     scheduled_end       TIMESTAMP NOT NULL,
89     session_type        VARCHAR(20) NOT NULL DEFAULT 'lecture',
90     status              VARCHAR(20) NOT NULL DEFAULT 'scheduled',
91     created_at          TIMESTAMP NOT NULL DEFAULT NOW()
92 );
93
94 CREATE TABLE image_queue (
95     id                  SERIAL PRIMARY KEY,
96     esp32_device_id     INTEGER REFERENCES esp32_devices(id),
97     session_id          INTEGER REFERENCES class_sessions(id),
98     image_path          VARCHAR(500) NOT NULL,
99     captured_at         TIMESTAMP NOT NULL,
100    processed_at        TIMESTAMP,
101    status              VARCHAR(20) NOT NULL DEFAULT 'pending',
102    error_msg           TEXT,
103    created_at          TIMESTAMP NOT NULL DEFAULT NOW()
104 );
105
106 CREATE TABLE attendance_records (
107     id                  SERIAL PRIMARY KEY,
108     session_id          INTEGER NOT NULL REFERENCES class_sessions
109         (id),
```

```

109  student_id      INTEGER NOT NULL REFERENCES students(id),
110  status          VARCHAR(20) NOT NULL DEFAULT 'present',
111  method          VARCHAR(20) NOT NULL DEFAULT '
    face_recognition',
112  confidence_score REAL,
113  image_queue_id  INTEGER REFERENCES image_queue(id),
114  marked_by       INTEGER REFERENCES staff(id),
115  created_at      TIMESTAMP NOT NULL DEFAULT NOW(),
116  UNIQUE (session_id, student_id)
117 );
118
119 CREATE TABLE student_embeddings (
120  id              SERIAL PRIMARY KEY,
121  student_id      INTEGER NOT NULL REFERENCES students(id)
    ,
122  embedding       BYTEA NOT NULL,
123  model_version   VARCHAR(50) NOT NULL,
124  demographic_group VARCHAR(50),
125  active          BOOLEAN NOT NULL DEFAULT TRUE,
126  created_at      TIMESTAMP NOT NULL DEFAULT NOW()
127 );
128
129 CREATE TABLE fairness_metrics (
130  id              SERIAL PRIMARY KEY,
131  test_date       DATE NOT NULL,
132  dataset         VARCHAR(100) NOT NULL DEFAULT 'FairFace',
133  race_group      VARCHAR(50) NOT NULL,
134  sample_size     INTEGER,
135  detection_rate  REAL,
136  precision       REAL,
137  recall          REAL,
138  f1              REAL,
139  threshold_used  REAL,
140  notes           TEXT,
141  created_at      TIMESTAMP NOT NULL DEFAULT NOW()
142 );
143
144 CREATE TABLE audit_log (
145  id              SERIAL PRIMARY KEY,
146  staff_id        INTEGER REFERENCES staff(id),
147  action          VARCHAR(50) NOT NULL,

```

```
148     target_table VARCHAR(50) NOT NULL ,
149     target_id    INTEGER ,
150     old_value    JSONB ,
151     new_value    JSONB ,
152     created_at   TIMESTAMP NOT NULL DEFAULT NOW()
153 );
154
155 -- Indexes
156 CREATE INDEX idx_attendance_session ON attendance_records(
157     session_id);
158 CREATE INDEX idx_attendance_student ON attendance_records(
159     student_id, created_at);
160 CREATE INDEX idx_queue_status      ON image_queue(status ,
161     created_at);
162 CREATE INDEX idx_embeddings_model  ON student_embeddings(
163     model_version, active);
164 CREATE INDEX idx_sessions_module  ON class_sessions(
165     module_id, scheduled_start);
166 CREATE INDEX idx_enrollments_lookup ON enrollments(student_id
167     , module_id);
```